

Отчет по лабораторной работе №4

Егор Витальевич Кузьмин

Содержание

1	Цель работы	6
2	Постановка задачи	7
3	Теоретические сведения	8
3.1	Агентное моделирование	8
3.2	Модель SIR	8
4	Выполнение работы	9
4.1	Инициализация проекта	9
4.2	Базовый скрипт модели SIR	10
4.3	Параметризованная версия базового скрипта	19
4.4	Скрипт параметрического исследования коэффициента заражения	29
4.5	Параметризованная версия второго скрипта	45
4.6	Скрипт исследования миграции	55
4.7	Параметризованная версия скрипта миграции	70
4.8	Скрипт оптимизации параметров модели	82
4.9	Параметризованная версия скрипта оптимизации	92
4.10	Скрипт комплексной визуализации результатов	99
5	Дополнительные задания	107
5.1	Дополнительные задания 1–3	107
5.2	Дополнительные задания 4–5	120
5.3	Дополнительное задание 6	134
6	Итоговые выводы	143
7	Список литературы	144

Список иллюстраций

4.1	Проверка установленных пакетов Julia и создание первого файла . . .	9
4.2	Выполнение первого скрипта, создание literate-версии и генерация необходимых файлов	13
4.3	Notebook literate-версии первого скрипта	13
4.4	График динамики базовой модели SIR	14
4.5	Notebook параметризованной literate-версии первого скрипта	20
4.6	Сравнение сценариев по доле инфицированных	21
4.7	Сравнение сценариев по доле выздоровевших	22
4.8	Выполнение второго скрипта в обычном формате	35
4.9	Notebook literate-версии второго скрипта	35
4.10	Таблица результатов параметрического исследования	36
4.11	Зависимость характеристик эпидемии от коэффициента заражения .	37
4.12	Создание параметрической версии второго скрипта и генерация файлов	45
4.13	Notebook параметризованной версии второго скрипта	46
4.14	Сценарное исследование смертности	46
4.15	Сценарное исследование пика эпидемии	47
4.16	Создание и выполнение третьего скрипта, подготовка literate-версии	60
4.17	Notebook literate-версии третьего скрипта	61
4.18	Влияние интенсивности миграции на динамику эпидемии	62
4.19	Создание параметрической версии третьего скрипта	70
4.20	Notebook параметрической версии третьего скрипта	71
4.21	Сценарное исследование времени до пика при миграции	72
4.22	Сценарное исследование пикового уровня инфицирования при миграции	73
4.23	Создание и выполнение четвёртого скрипта, подготовка literate- версии	85
4.24	Notebook literate-версии четвёртого скрипта	86
4.25	Генерация файлов и notebook для параметризованной версии четвёртого скрипта	92
4.26	Выполненная параметризованная версия четвёртого скрипта	92
4.27	Создание и выполнение пятого скрипта, генерация literate-версии и notebook	101
4.28	Комплексный анализ результатов моделирования	102
5.1	Создание и выполнение скрипта дополнительных заданий 1–3 . . .	116

5.2	Базовая динамика SIR для дополнительных заданий	117
5.3	Исследование порога эпидемии	118
5.4	Динамика инфицированных по городам при гетерогенных параметрах	119
5.5	Общая динамика при гетерогенных параметрах	120
5.6	Создание и выполнение скрипта дополнительных заданий 4–5 . . .	132
5.7	Сравнение времени достижения пика с карантином и без карантина	132
5.8	Сравнение величины пика с карантином и без карантина	133
5.9	Проверка оптимального решения для дополнительного задания 6 . .	142

Список таблиц

1 Цель работы

Целью лабораторной работы является изучение реализации эпидемиологической модели SIR в агентном подходе, исследование влияния параметров на динамику распространения инфекции, освоение литературного программирования с использованием пакета `Literate.jl`, а также выполнение дополнительных вычислительных экспериментов, связанных с порогом эпидемии, гетерогенностью, миграцией, карантинными мерами и оптимизацией параметров модели.

2 Постановка задачи

В ходе выполнения лабораторной работы требовалось:

1. реализовать базовую модель SIR в агентном подходе;
2. выполнить запуск скриптов в обычном формате;
3. оформить решения в literate-стиле;
4. сгенерировать notebook и документацию Quarto;
5. выполнить параметрические исследования;
6. проанализировать влияние миграции;
7. исследовать эффекты карантинных мер;
8. решить задачу оптимизации параметров;
9. оформить результаты в виде отчёта.

3 Теоретические сведения

3.1 Агентное моделирование

Агентное моделирование представляет систему как совокупность взаимодействующих агентов. Каждый агент описывается собственным состоянием и набором правил поведения. Глобальная динамика системы формируется как результат большого числа локальных взаимодействий.

3.2 Модель SIR

Модель SIR описывает распространение инфекционного заболевания через три состояния агента:

- S — восприимчивый;
- I — инфицированный;
- R — выздоровевший.

В агентной постановке каждый человек задаётся отдельным агентом. В рассматриваемой лабораторной работе учитываются заражение, выздоровление, повторное заражение, смертность и перемещение агентов между городами.

4 Выполнение работы

4.1 Инициализация проекта

На первом этапе была выполнена активация проекта, проверка установленных библиотек Julia и подготовка первого рабочего файла.

```
evkuzmin@fedora ~$ julia --help
Manifest No packages added to or removed from "/work/study/2026-1/simulation-modeling/labs/lab04/scripts/Manifest.toml"

julia> Pkg.status()
Status "/work/study/2026-1/simulation-modeling/labs/lab04/scripts/Project.toml"
  [4606a434] Agents v7.0.0
  [56b0baf2] BenchmarkTools v1.6.3
  [a1344002] BlackBoxOptim v0.6.4
  [336ed08f] CSV v0.10.16
  [1373f908] CairoMakie v0.15.0
  [29595af9] DataFrames v1.7.1
  [0c460032] DifferentialEquations v7.17.0
  [31c24e10] Distributions v0.25.123
  [83403008] DrWatson v2.19.1
  [7a2cc8ca] FFTW v1.16.0
  [7873ff75] IDJulia v1.26.0
  [0338350b] JLD2 v0.6.3
  [89064f2f] LaTeXStrings v1.4.0
  [98e91d3d] Iterate v2.21.0
  [91a5bc0d] Plots v1.40.20
  [d7187b65] Quarto v1.0.0
  [8950c32a] SimpleDiffEq v1.13.0
  [2913b802] StatsBase v0.34.10
  [f3b20707] StatsPlots v0.15.8
  [bd369af6] Tables v1.12.1
  [8a7ff205] Random v1.11.0
Info Packages marked with * have new versions available and may be upgradable.

julia> exit()
[evkuzmin@fedora scripts]$ cd ..
[evkuzmin@fedora lab04]$ cd src
[evkuzmin@fedora src]$ touch sir_model.jl
[evkuzmin@fedora src]$ gedit sir_model.jl
[evkuzmin@fedora src]$
```

Рисунок 4.1: Проверка установленных пакетов Julia и создание первого файла

На рисунке показан этап инициализации проекта. Видно, что окружение Julia активировано корректно и доступны необходимые библиотеки для выполнения лабораторной работы.

4.2 Базовый скрипт модели SIR

4.2.1 Обычная версия

Для начала была подготовлена и выполнена обычная версия первого скрипта, реализующего базовую динамику модели SIR.

Листинг 4.1. Обычная версия базового скрипта

sir_run_basic.jl

```
using DrWatson
@quickactivate "project"

using Agents, DataFrames, Plots
using JLD2

include(sourcedir("sir_model.jl"))

#  

params = Dict(
    :Ns => [1000, 1000, 1000],
    :_und => [0.5, 0.5, 0.5],
    :_det => [0.05, 0.05, 0.05],
    :infection_period => 14,
    :detection_time => 7,
    :death_rate => 0.02,
    :reinfection_probability => 0.1,
    :Is => [0, 0, 1],
    :seed => 42,
    :n_steps => 100,
)
```

```

# Initialize the model
model = initialize_sir(; params...)

# Create arrays to store the results
times = Int[]
S_vals = Int[]
I_vals = Int[]
R_vals = Int[]
total_vals = Int[]

# Iterate over the steps
for step = 1:params[:n_steps]
    Agents.step!(model, 1)
    push!(times, step)
    push!(S_vals, susceptible_count(model))
    push!(I_vals, infected_count(model))
    push!(R_vals, recovered_count(model))
    push!(total_vals, total_count(model))
end

# Create a DataFrame to store the results
agent_df = DataFrame(
    time = times,
    susceptible = S_vals,
    infected = I_vals,
    recovered = R_vals,
)

```

```

model_df = DataFrame(
    time = times,
    total = total_vals,
)

# 
plot(
    agent_df.time,
    agent_df.susceptible,
    label = ",
    xlabel = ",
    ylabel = ",
)
plot!(agent_df.time, agent_df.infected, label = ")
plot!(agent_df.time, agent_df.recovered, label = ")
plot!(agent_df.time, model_df.total, label = " ( )",

savefig(plotsdir("sir_basic_dynamics.png"))

#  
@save datadir("sir_basic_agent.jld2") agent_df
@save datadir("sir_basic_model.jld2") model_df

```

4.2.2 Выполнение и генерация literate-версии

После запуска обычной версии была сформирована literate-версия и сгенерированы производные файлы: чистый код, notebook и Quarto-документация.

```

sir_basic_dynamics.png
[evkuzn@fedora plots]$ cd ..
[evkuzn@fedora scripts]$ cd data
[evkuzn@fedora data]$ ls
sir_basic_agent.jld2 sir_basic_model.jld2
[evkuzn@fedora data]$ cd ..
[evkuzn@fedora scripts]$ ls
data Manifest.toml plots Project.toml sir_run_basic.jl src
[evkuzn@fedora scripts]$ touch sir_literate.jl
[evkuzn@fedora scripts]$ gedit sir_literate.jl
[evkuzn@fedora scripts]$ ls
data Manifest.toml plots Project.toml sir_run_basic.jl src tangle.jl
[evkuzn@fedora scripts]$ julia --project= tangle.jl sir_literate.jl
[ Info: generating plain script file from "~/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scripts/sir_literate.jl"
[ Info: writing result to "~/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scripts/sir_literate/sir_literate.jl"
✓ Чистый экспорт: /home/evkuzmin/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scripts/sir_literate/sir_literate.jl
[ Info: generating markdown page from "~/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scripts/sir_literate.jl"
[ Info: writing result to "~/work/study/2026-1/simulation-modeling/labs/lab04/scripts/markdown/sir_literate/sir_literate.qmd"
✓ Quarto: /home/evkuzmin/work/study/2026-1/simulation-modeling/labs/lab04/scripts/markdown/sir_literate/sir_literate.qmd
[ Info: generating notebook from "~/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scripts/sir_literate.jl"
[ Info: writing result to "~/work/study/2026-1/simulation-modeling/labs/lab04/scripts/notebooks/sir_literate/sir_literate.ipynb"
✓ Notebook: /home/evkuzmin/work/study/2026-1/simulation-modeling/labs/lab04/scripts/notebooks/sir_literate/sir_literate.ipynb

Готово! Все файлы созданы.
[evkuzn@fedora scripts]$ ls
data Manifest.toml markdown notebooks plots Project.toml scripts sir_literate.jl sir_run_basic.jl src tangle.jl
[evkuzn@fedora scripts]$ ls notebooks
sir_literate
[evkuzn@fedora scripts]$ ls
data Manifest.toml markdown notebooks plots Project.toml scripts sir_literate.jl sir_run_basic.jl src tangle.jl
[evkuzn@fedora scripts]$ ls notebooks/sir_literate
sir_literate.ipynb
[evkuzn@fedora scripts]$ ls plots
sir_basic_dynamics.png
[evkuzn@fedora scripts]$ ls markdown/sir_literate
sir_literate.qmd
[evkuzn@fedora scripts]$ jupyter notebook

```

Рисунок 4.2: Выполнение первого скрипта, создание literate-версии и генерация необходимых файлов

На рисунке показан этап выполнения первого скрипта и генерации literate-артефактов. Также видна проверка того, что необходимые файлы были созданы.

4.2.3 Notebook literate-версии первого скрипта

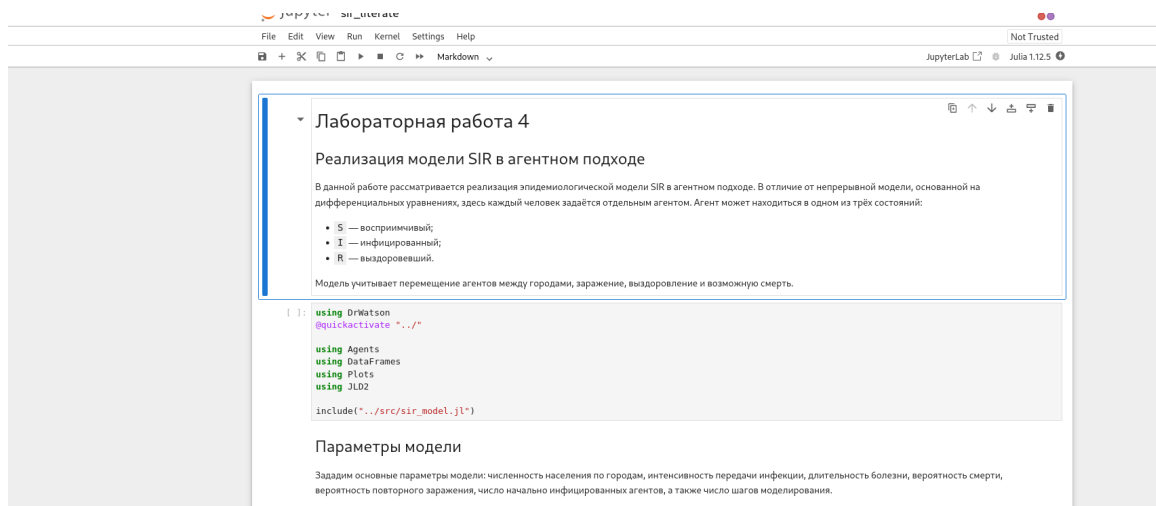


Рисунок 4.3: Notebook literate-версии первого скрипта

На рисунке представлен notebook, полученный из literate-версии первого скрипта. Это подтверждает корректность преобразования исходного файла в

формат документации.

4.2.4 Результат выполнения базовой модели

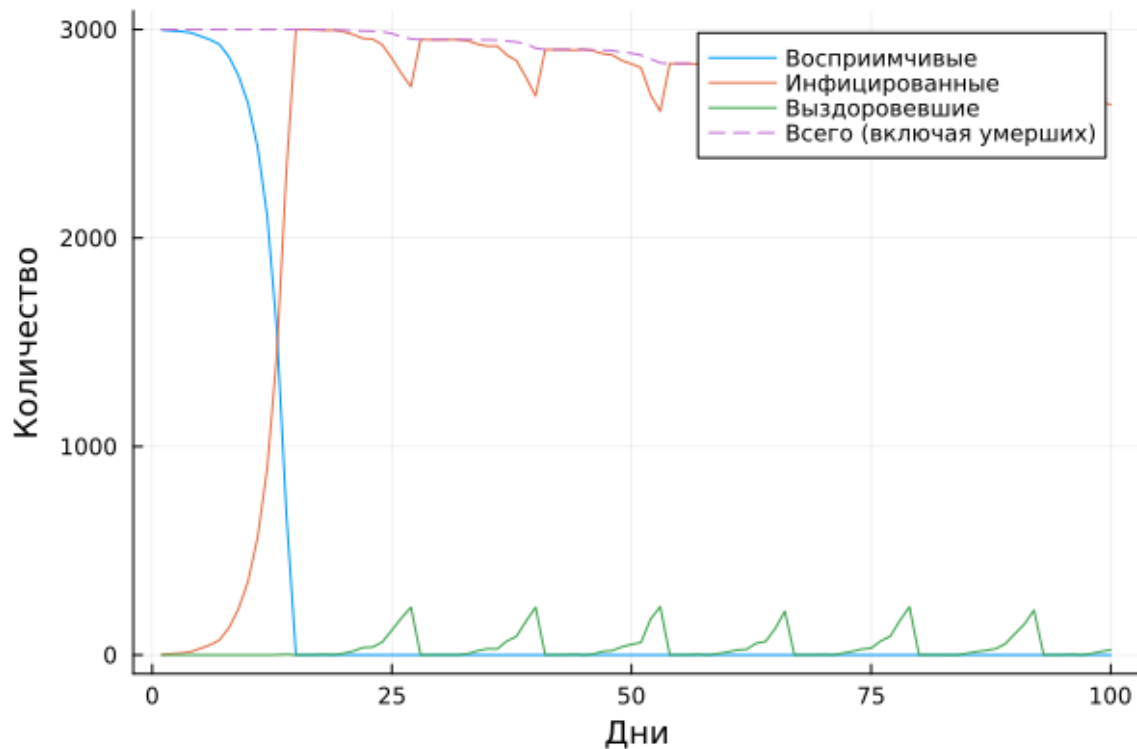


Рисунок 4.4: График динамики базовой модели SIR

На рисунке представлена динамика числа восприимчивых, инфицированных и выздоровевших агентов в базовой модели. В начале моделирования число инфицированных растёт, затем после достижения максимума начинает уменьшаться. Одновременно сокращается число восприимчивых агентов и увеличивается число выздоровевших, что соответствует классической интерпретации модели SIR.

4.2.5 Literate-версия базового скрипта

Листинг 4.2. Literate-версия базового скрипта

sir_literate.jl

```
# # 4
# ## SIR
#
# SIR
# .
# ,
# .
# :
#
# - `S` - ;
# - `I` - ;
# - `R` - .
#
# ,
# .

using DrWatson
@quickactivate "../"

using Agents
using DataFrames
using Plots
using JLD2

include("../src/sir_model.jl")
```

```

# ## 初始化参数
#
# 初始化参数: 初始化参数,
# 初始化参数, 初始化参数, 初始化参数,
# 初始化参数, 初始化参数, 初始化参数,
# 初始化参数, 初始化参数, 初始化参数,
# 初始化参数, 初始化参数, 初始化参数.

params = Dict(
    :Ns => [1000, 1000, 1000],
    :xi_und => [0.5, 0.5, 0.5],
    :xi_det => [0.05, 0.05, 0.05],
    :infection_period => 14,
    :detection_time => 7,
    :death_rate => 0.02,
    :reinfection_probability => 0.1,
    :Is => [0, 0, 1],
    :seed => 42,
    :n_steps => 100,
)

# ## 初始化模型
#
# 初始化模型, 初始化模型, 初始化模型.

model = initialize_sir(; params...)

# ## 初始化模型
#

```



```
# 定义模型参数
#
# - 初始时间;
# - 初始易感者数量;
# - 初始感染者数量;
# - 初始康复者数量;
# - 初始总人口数。

times = Int[]
S_vals = Int[]
I_vals = Int[]
R_vals = Int[]
total_vals = Int[]

# ## 模型初始化
#
# 初始化模型参数
# 初始化模型参数。

for step = 1:params[:n_steps]
    Agents.step!(model, 1)
    push!(times, step)
    push!(S_vals, susceptible_count(model))
    push!(I_vals, infected_count(model))
    push!(R_vals, recovered_count(model))
    push!(total_vals, total_count(model))
end
```

```

# ## 1000000000 100000 1000000000
#
# 1000 1000000000 10000000000 10000000 10000000 1000000000 1000000 1 10000 1000000.

agent_df = DataFrame(
    time = times,
    susceptible = S_vals,
    infected = I_vals,
    recovered = R_vals,
)

model_df = DataFrame(
    time = times,
    total = total_vals,
)

# ## 10000000000 10000000000
#
# 1000000000 1000000 1000000000 10000000000 100000 100 10000000.

plot(
    agent_df.time,
    agent_df.susceptible,
    label = "1000000000000000",
    xlabel = "1000",
    ylabel = "10000000000",
)

plot!(agent_df.time, agent_df.infected, label = "1000000000000000")

```

```

plot!(agent_df.time, agent_df.recovered, label = "Recovered")
plot!(agent_df.time, model_df.total, label = "Model (Susceptible + Exposed + Infected)",

savefig(plotsdir("sir_basic_dynamics_literate.png"))

# ## Susceptible Population
#
# The susceptible population decreases over time as individuals
# become infected. The model uses the JLD2 format.

@save datadir("sir_basic_agent_literate.jld2") agent_df
@save datadir("sir_basic_model_literate.jld2") model_df

# ## Results
#
# The following plots show the results of the simulation. The
# top plot shows the susceptible population (blue line) and the
# total population (red line) over time. The bottom plot shows
# the infected population (green line) over time.

```

4.3 Параметризованная версия базового скрипта

Следующим этапом была подготовлена параметризованная literate-версия первого скрипта.

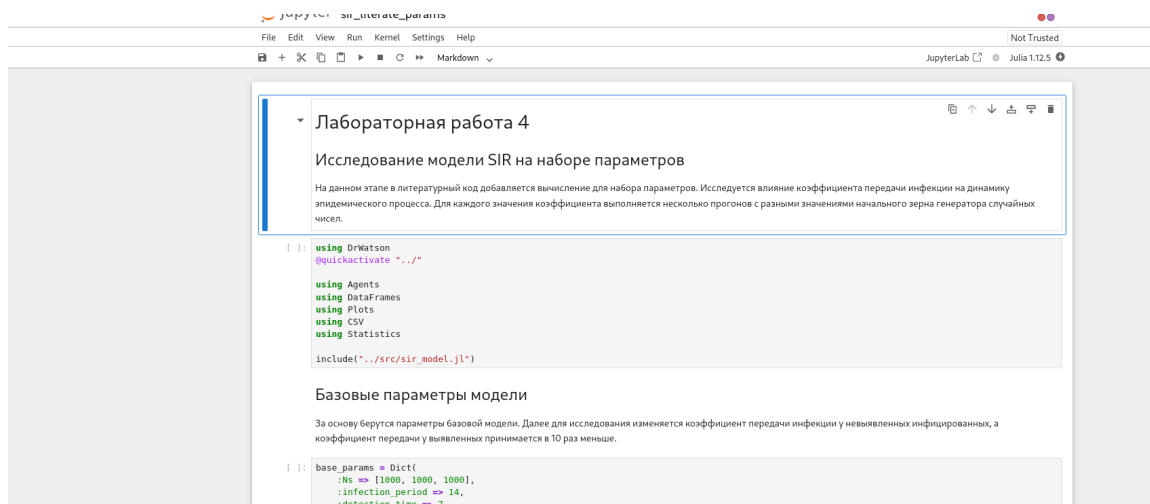


Рисунок 4.5: Notebook параметризованной literate-версии первого скрипта

На рисунке показан notebook параметризованной версии первого скрипта.

4.3.1 Результаты параметризованной версии

Сравнение динамики инфицированных для разных параметров

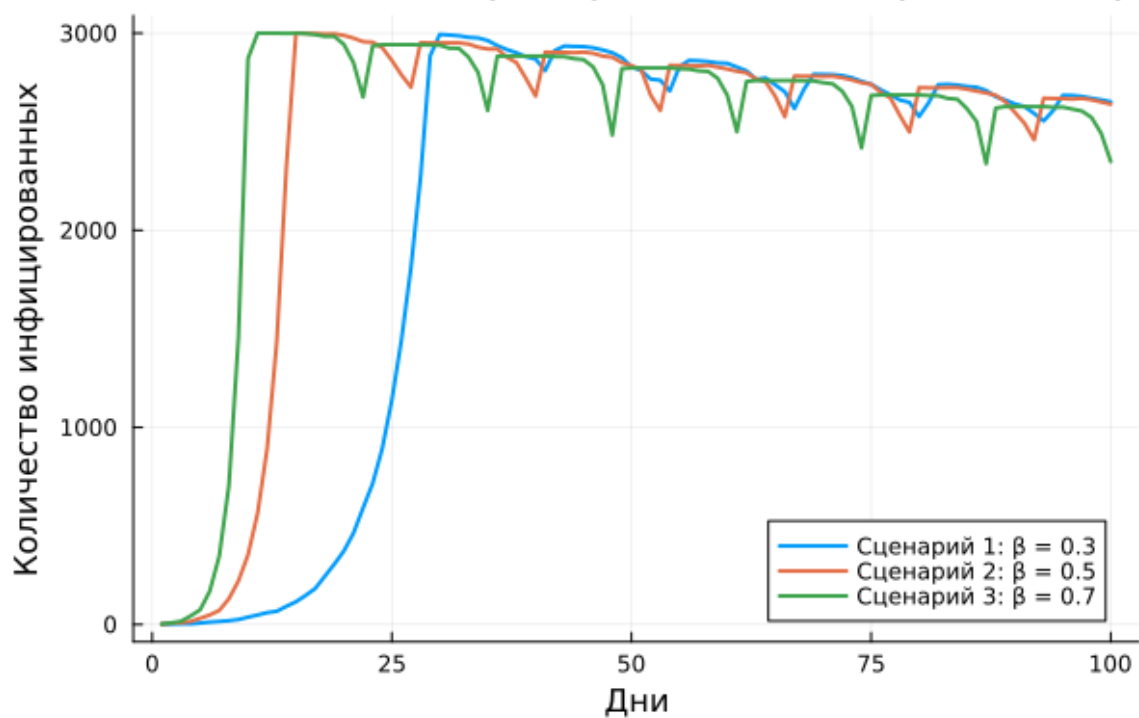


Рисунок 4.6: Сравнение сценариев по доле инфицированных

На рисунке сравниваются сценарии по доле инфицированных. Видно, что более высокий коэффициент заражения ускоряет развитие эпидемии и увеличивает максимум числа инфицированных.

Сравнение динамики выздоровевших для разных параметров

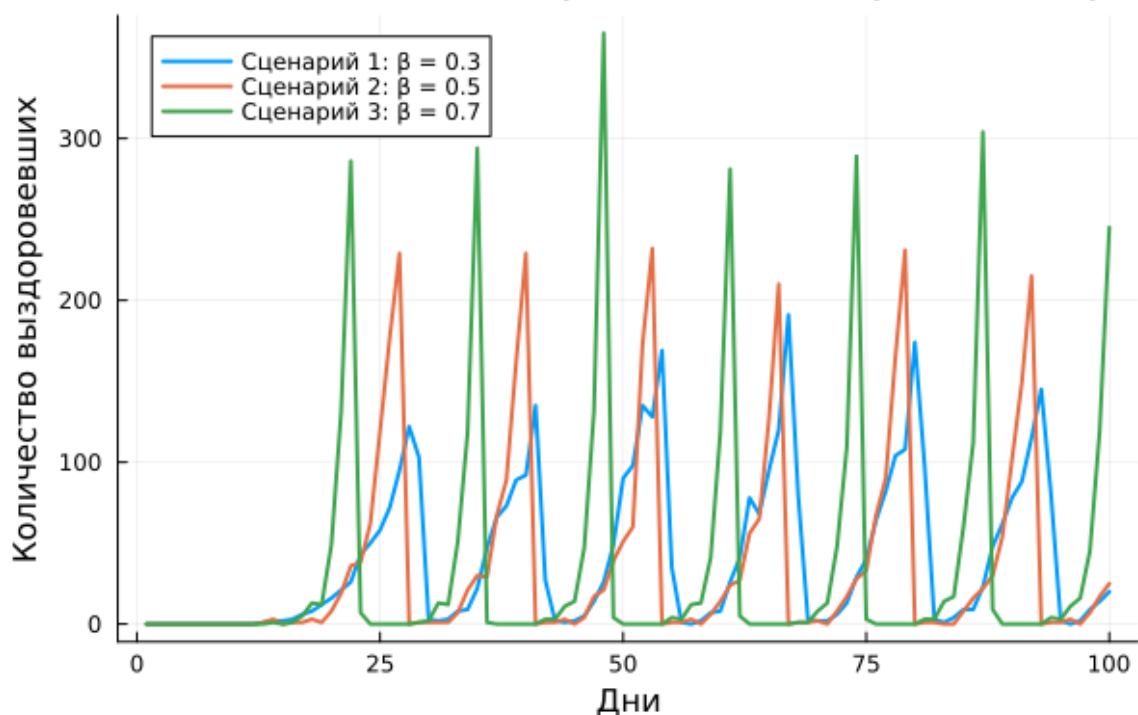


Рисунок 4.7: Сравнение сценариев по доле выздоровевших

На рисунке показана динамика выздоровевших для различных параметров. При более интенсивном распространении инфекции число выздоровевших также растёт быстрее.

Листинг 4.3. Параметризованная literate-версия базового скрипта

`sir_literate_params.jl`

```
# # 4
# ## SIR
# ##
#
# 4,
# .
#
```

```

# 1x1 1000x1000 1000x1000 1000x1000x1000 1000x1000 1000x1000.

using DrWatson
@quickactivate "../"

using Agents
using DataFrames
using Plots
using JLD2

include("../src/sir_model.jl")

# ## 1000x1000 1000x1000x1000
#
# 1000x1000 1000x1000x1000, 1000x1000 1000x1000x1000 1000 1000 1000x1000x1000.

base_params = Dict(
    :Ns => [1000, 1000, 1000],
    :infection_period => 14,
    :detection_time => 7,
    :death_rate => 0.02,
    :reinfection_probability => 0.1,
    :Is => [0, 0, 1],
    :seed => 42,
    :n_steps => 100,
)

# ## 1000x1000 1000x1000x1000

```



```

)

model = initialize_sir(; params...)

times = Int[]
S_vals = Int[]
I_vals = Int[]
R_vals = Int[]
total_vals = Int[]

for step = 1:params[:n_steps]
    Agents.step!(model, 1)
    push!(times, step)
    push!(S_vals, susceptible_count(model))
    push!(I_vals, infected_count(model))
    push!(R_vals, recovered_count(model))
    push!(total_vals, total_count(model))
end

agent_df = DataFrame(
    time = times,
    susceptible = S_vals,
    infected = I_vals,
    recovered = R_vals,
    scenario = fill(scenario_name, length(times)),
)

model_df = DataFrame(

```

```

        time = times,
        total = total_vals,
        scenario = fill(scenario_name, length(times)),
    )

    return agent_df, model_df
end

# ## 运行场景并返回结果
#
# 运行场景并返回结果
# 运行场景并返回结果。

all_agent_df = DataFrame()
all_model_df = DataFrame()

for p in param_sets
    agent_df, model_df = run_scenario(base_params, p._und, p._det,
    append!(all_agent_df, agent_df)
    append!(all_model_df, model_df)
end

# ## 运行场景并返回结果
#
# 运行场景并返回结果
# 运行场景并返回结果。

first(all_agent_df, 10)

```

```

# ## Plot of the number of infected individuals over time
#
# Plot of the number of infected individuals over time for each scenario.

plot(
  xlabel = "Time",
  ylabel = "Number of infected individuals",
  title = "Number of infected individuals over time for each scenario",
)

for p in param_sets
  subset_df = all_agent_df[all_agent_df.scenario == p.name, :]
  plot!(
    subset_df.time,
    subset_df.infected,
    label = p.name,
    lw = 2,
  )
end

savefig(plotsdir("sir_literate_params_infected.png"))

# ## Plot of the number of infected individuals over time
#
# Plot of the number of infected individuals over time for each scenario.

plot(
  xlabel = "Time",

```

```

        ylabel = "Recovered",
        title = "Recovered vs Time for Scenario " & p.name,
    )

for p in param_sets
    subset_df = all_agent_df[all_agent_df.scenario .== p.name, :]
    plot!(
        subset_df.time,
        subset_df.recovered,
        label = p.name,
        lw = 2,
    )
end

savefig(plotsdir("sir_literate_params_recovered.png"))

# ## Parameters
#
# Parameters for the SIR model. JLD2.

@save datadir("sir_literate_params_agent.jld2") all_agent_df
@save datadir("sir_literate_params_model.jld2") all_model_df

# ## Results
#
# The figure shows the time evolution of the SIR model. The x-axis is time (days) and the y-axis is the number of individuals in each compartment. The legend indicates the compartments: S (Susceptible), I (Infected), and R (Recovered).
# The plot shows that the number of susceptible individuals decreases over time, while the number of infected individuals increases and then decreases. The number of recovered individuals increases over time.
# The plot also shows that the total number of individuals remains constant over time, as expected for a closed system.

```

```
# Задаем значения параметров модели.
```

4.4 Скрипт параметрического исследования коэффициента заражения

Во втором скрипте реализуется перебор значений коэффициента заражения β и вычисление ключевых характеристик эпидемического процесса.

4.4.1 Обычная версия

Листинг 4.4. Обычная версия скрипта параметрического исследования
sir_scan_beta.jl

```
using DrWatson
@quickactivate "project"

using Agents, DataFrames, Plots, CSV, Random
using Statistics

include(scrdir("sir_model.jl"))

# Задаем значения параметров модели.
function run_experiment(p)
    # Задаем значения параметров модели.
    beta = p[:beta]
    n_und = fill(beta, 3)
    n_det = fill(beta / 10, 3)
```

```

# Initialize the model
model = initialize_sir(
    Ns = p[:Ns],
    beta_und = beta_und,
    beta_det = beta_det,
    infection_period = p[:infection_period],
    detection_time = p[:detection_time],
    death_rate = p[:death_rate],
    reinfection_probability = p[:reinfection_probability],
    Is = p[:Is],
    seed = p[:seed],
    n_steps = p[:n_steps],
)

# Calculate the infected fraction at each time step
infected_fraction(model) =
    count(a.status == :I for a in allagents(model)) / nagents(model)

peak_infected = 0.0

# Iterate over the simulation steps
for step = 1:p[:n_steps]
    # Calculate the infected fraction at each time step
    agent_ids = collect(allids(model))
    for id in agent_ids
        agent = try
            model[id]
        catch
    end
end

```

```

nothing
end

if agent != nothing
  sir_agent_step!(agent, model)
end
end

frac = infected_fraction(model)
if frac > peak_infected
  peak_infected = frac
end
end

# XXXXXXXX XXXXXXXX

final_infected = infected_fraction(model)
final_recovered = count(a.status == :R for a in allagents(model)) /
total_deaths = sum(p[:Ns]) - nagents(model)

return (
  peak = peak_infected,
  final_inf = final_infected,
  final_rec = final_recovered,
  deaths = total_deaths,
)
end

# XXXXXXXX XXXXXXXX beta

```

```

beta_range = 0.1:0.1:1.0

# 初始化 seed 和 参数列表
seeds = [42, 43, 44]

# 初始化 参数列表
params_list = []

for b in beta_range
  for s in seeds
    push!(
      params_list,
      Dict(
        :beta => b,
        :Ns => [1000, 1000, 1000],
        :infection_period => 14,
        :detection_time => 7,
        :death_rate => 0.02,
        :reinfection_probability => 0.1,
        :Is => [0, 0, 1],
        :seed => s,
        :n_steps => 100,
      ),
    )
  end
end

# 初始化 结果 和 参数列表

```



```

results = []

for params in params_list
  data = run_experiment(params)
  push!(results, merge(params, Dict(pairs(data))))
  println("XXXXXXXXX XXXXXXXXXXXX X beta = $(params[:beta]), seed = $(params
end

# XXXXXXXXXXXX XXXXXXXXXXXX XXXX XXXXXXXX

df = DataFrame(results)
CSV.write(datadir("beta_scan_all.csv"), df)

# XXXXXXXXXXXX XXXXXXXXXXXX XX XXXXXXXXXXXX XXXXXXXXXXXX beta

grouped = combine(
  groupby(df, [:beta]),
  :peak => mean => :mean_peak,
  :final_inf => mean => :mean_final_inf,
  :deaths => mean => :mean_deaths,
)

# XXXXXXXXXXXX XXXXXXXXXXXX XXXXXXXX

plot(
  grouped.beta,
  grouped.mean_peak,
  label = "XX XXXXXXXX",
  xlabel = "XXXXXXXXXXXX XXXXXXXXXXXX X",
  ylabel = "XXX XXXXXXXXXXXXXXXX",
  marker = :circle,

```

```

linewidth = 2,
)

plot!(
    grouped.beta,
    grouped.mean_final_inf,
    label = "Infected cases",
    marker = :square,
)

plot!(
    grouped.beta,
    grouped.mean_deaths ./ 3000,
    label = "Deaths",
    marker = :diamond,
)

savefig(plotsdir("beta_scan.png"))

println("Infected cases and deaths data from data/beta_scan_all.csv saved to plots/beta_scan.png")

```

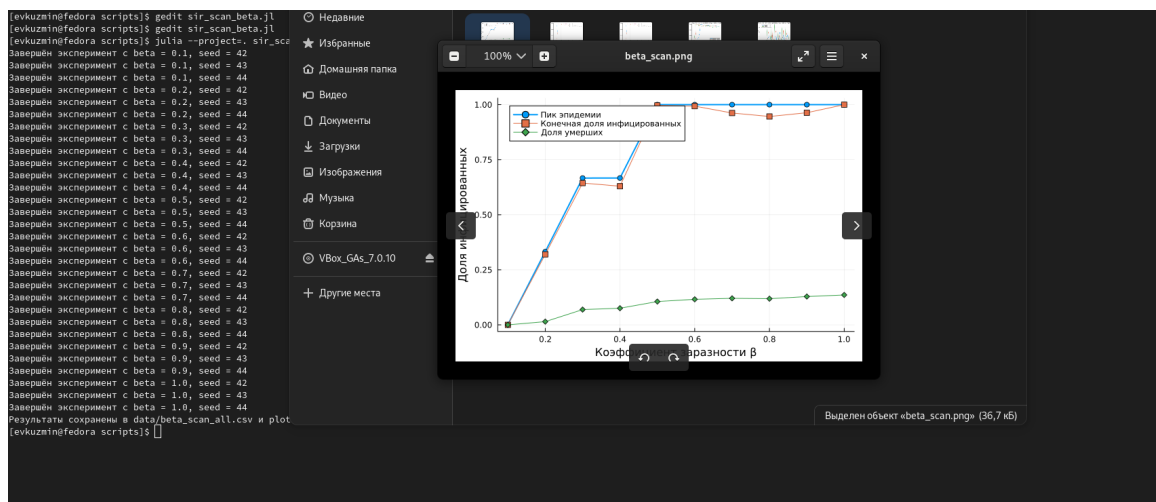


Рисунок 4.8: Выполнение второго скрипта в обычном формате

На рисунке показан запуск второго скрипта в обычной версии.

4.4.2 Notebook literate-версии

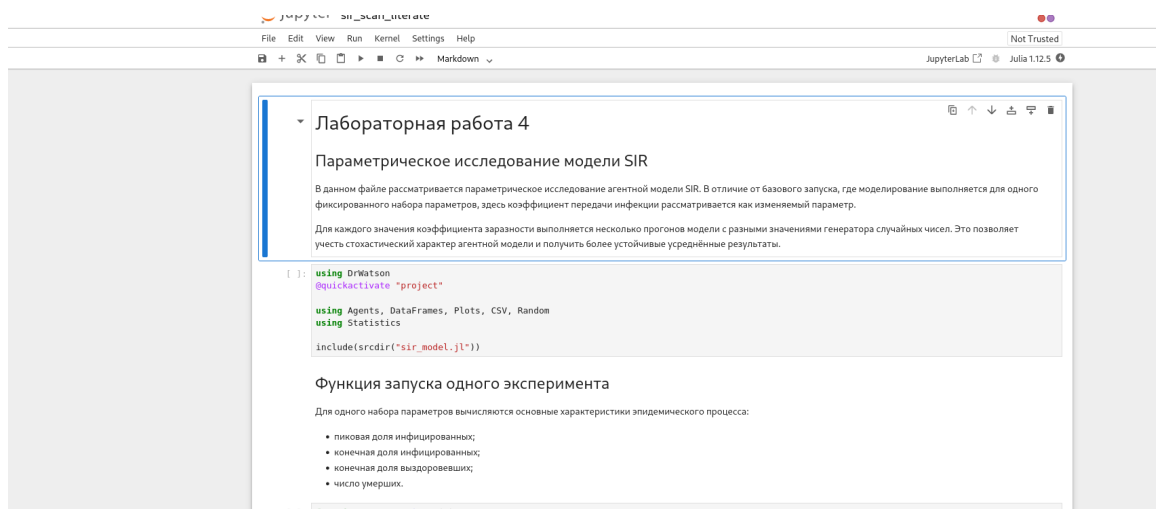


Рисунок 4.9: Notebook literate-версии второго скрипта

На рисунке представлен notebook literate-версии второго скрипта.

4.4.3 Генерация notebook и документации

```
[evkuzmin@fedora scripts]$ touch sir_scan_literate.jl
[evkuzmin@fedora scripts]$ gedit sir_scan_literate.jl
[evkuzmin@fedora scripts]$ julia --project=. tangle.jl sir_scan_literate.jl
Info: generating plain script file from "~/work/study/2026-1/simulation-modeling/labs/lab04/scripts/sir_scan_literate.jl"
Info: writing result to "~/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scripts/sir_scan_literate/sir_scan_literate.jl"
✓ Чистый скрипт: /home/evkuzmin/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scripts/sir_scan_literate/sir_scan_literate.jl

Info: generating markdown page from "~/work/study/2026-1/simulation-modeling/labs/lab04/scripts/sir_scan_literate.jl"
Info: writing result to "~/work/study/2026-1/simulation-modeling/labs/lab04/scripts/markdown/sir_scan_literate/sir_scan_literate.qmd"
✓ Quarto: /home/evkuzmin/work/study/2026-1/simulation-modeling/labs/lab04/scripts/markdown/sir_scan_literate/sir_scan_literate.qmd

Info: generating notebook from "~/work/study/2026-1/simulation-modeling/labs/lab04/scripts/sir_scan_literate.jl"
Info: writing result to "~/work/study/2026-1/simulation-modeling/labs/lab04/scripts/notebooks/sir_scan_literate/sir_scan_literate.ipynb"
✓ Notebook: /home/evkuzmin/work/study/2026-1/simulation-modeling/labs/lab04/scripts/notebooks/sir_scan_literate/sir_scan_literate.ipynb

Готово! Все файлы созданы.
[evkuzmin@fedora scripts]$
[evkuzmin@fedora scripts]$ ls notebooks/sir_scan_literate
sir_scan_literate.ipynb
[evkuzmin@fedora scripts]$ ls markdown/sir_scan_literate
sir_scan_literate.qmd
[evkuzmin@fedora scripts]$ ls
Manifest.toml  scripts  notebooks  plots  Project.toml  scan_literate_params.jl  scripts  sir_literate.jl  sir_literate_params.jl  sir_run_basic.jl  sir_scan_beta.jl  sir_scan_literate.jl  src  ta
```

На рисунке показан этап генерации notebook и Quarto-документации для literate-версии второго скрипта.

4.4.4 Таблица результатов

☐ Вычислять формулы

Поля

Тип столбца:

	Стандарт	Стандарт	Стандарт	Стандарт	Стандарт	Стандарт	Стандарт
15	100	0.00037023324694557573	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	14
16	100	0.0003729951510630362	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	14
17	100	0.014264264264264264	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14
18	100	0.0018782870022539444	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14
19	100	0.0038022813688212928	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14
20	100	0.09444872783346184	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14
21	100	0.002629601803155522	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14
22	100	0.016541353383458645	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14
23	100	0.13496932515337423	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	14
24	100	0.0023480220711502426	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	14

Справка Отменить OK

Рисунок 4.10: Таблица результатов параметрического исследования

На рисунке представлена таблица результатов параметрического исследования. Она содержит значения β и соответствующие показатели: пик эпидемии, конечную долю инфицированных, долю выздоровевших и число умерших.

4.4.5 Результат параметрического исследования

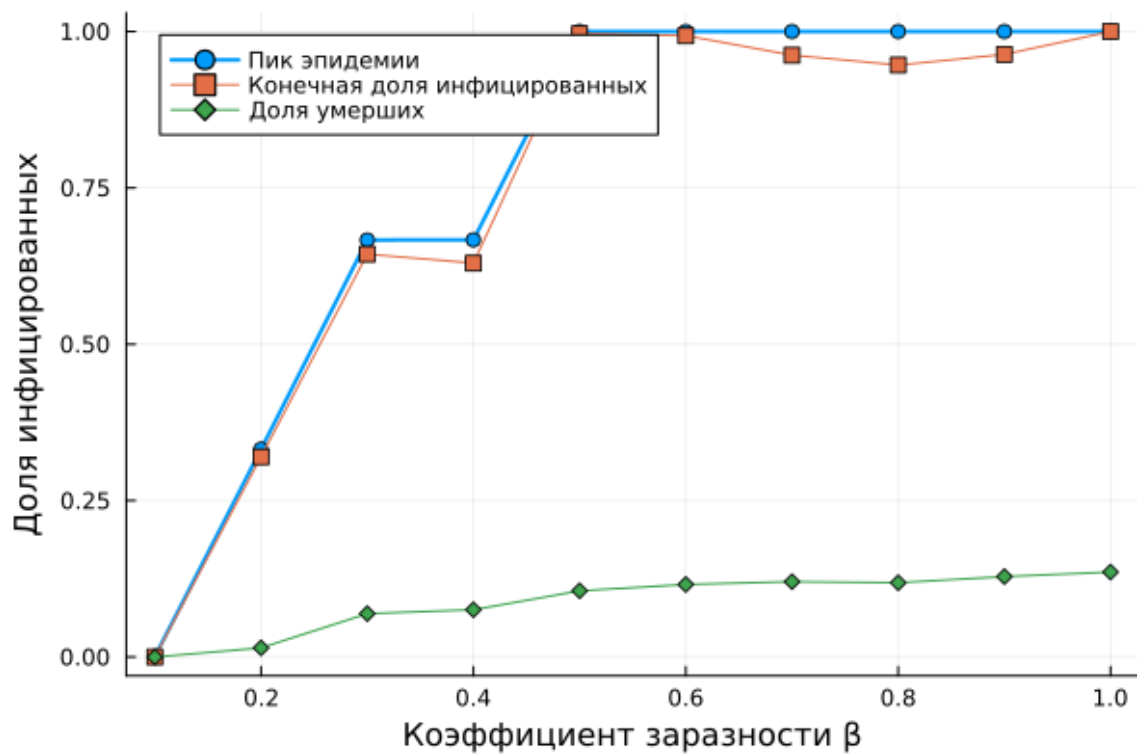


Рисунок 4.11: Зависимость характеристик эпидемии от коэффициента заражения

На рисунке показано влияние коэффициента заражения на основные характеристики эпидемии. С ростом β увеличиваются пик эпидемии, конечная доля инфицированных и смертность. Это соответствует ожидаемой интерпретации: более высокая заразность приводит к более интенсивному развитию эпидемического процесса.

4.4.6 Literate-версия второго скрипта

Листинг 4.5. Literate-версия скрипта параметрического исследования
sir_scan_literate.jl

```

# # 4
# ## SIR
#
# 4
# SIR. 4,
# 4,
# .
#
# 4
# 4
# .
# 4
# .
#
using DrWatson
@quickactivate "project"

using Agents, DataFrames, Plots, CSV, Random
using Statistics

include(sourcedir("sir_model.jl"))

# ##
#
# 4
# :
#
# - ;
# - ;

```

```

# - 初始化模型参数;
# - 初始化模型。

function run_experiment(p)
# 初始化 beta 参数
beta = p[:beta]
    und = fill(beta, 3)
    det = fill(beta / 10, 3)

# 初始化模型
model = initialize_sir(;
Ns = p[:Ns],
    und = und,
    det = det,
infection_period = p[:infection_period],
detection_time = p[:detection_time],
death_rate = p[:death_rate],
reinfection_probability = p[:reinfection_probability],
Is = p[:Is],
seed = p[:seed],
n_steps = p[:n_steps],
)

# 计算感染比例
infected_fraction(model) =
count(a.status == :I for a in allagents(model)) / nagents(model)

peak_infected = 0.0

```

```

# [comment] [comment]
for step = 1:p[:n_steps]
# [comment] [comment] [comment] [comment] [comment] [comment]
agent_ids = collect(allids(model))
for id in agent_ids
agent = try
model[id]
catch
nothing
end

if agent != nothing
sir_agent_step!(agent, model)
end
end

frac = infected_fraction(model)
if frac > peak_infected
peak_infected = frac
end
end

# [comment] [comment]
final_infected = infected_fraction(model)
final_recovered = count(a.status == :R for a in allagents(model)) /
total_deaths = sum(p[:Ns]) - nagents(model)

return (

```



```

peak = peak_infected,
final_inf = final_infected,
final_rec = final_recovered,
deaths = total_deaths,
)
end

# ## Parameter Estimation
#
# Estimate the parameters of the model beta seed 0.1 1.0
# 0.1. beta seed 0.1 1.0
# beta seed 0.1 1.0 seed.

beta_range = 0.1:0.1:1.0
seeds = [42, 43, 44]

# ## Parameter Estimation
#
# beta seed 0.1 1.0 beta seed 0.1 1.0
# beta seed 0.1 1.0.

params_list = []

for b in beta_range
for s in seeds
push!(
params_list,
Dict(

```

```

:beta => b,
:Ns => [1000, 1000, 1000],
:infection_period => 14,
:detection_time => 7,
:death_rate => 0.02,
:reinfection_probability => 0.1,
:Is => [0, 0, 1],
:seed => s,
:n_steps => 100,
),
)
end
end

# ##  
#
#      , 
#    .

results = []

for params in params_list
  data = run_experiment(params)
  push!(results, merge(params, Dict(pairs(data))))
  println("   beta = $(params[:beta]), seed = $(params[:seed])")
end

# ##  

```

```

#
# 读取所有数据并写入CSV文件
# 使用CSV格式。

df = DataFrame(results)
CSV.write(datadir("beta_scan_all.csv"), df)

# ## 计算每个beta值的统计量
#
# 计算每个beta值的统计量
# 计算每个beta值的统计量。

grouped = combine(
  groupby(df, [:beta]),
  :peak => mean => :mean_peak,
  :final_inf => mean => :mean_final_inf,
  :deaths => mean => :mean_deaths,
)

# ## 计算每个beta值的统计量
#
# 计算每个beta值的统计量
# 计算每个beta值的统计量。

plot(
  grouped.beta,
  grouped.mean_peak,
  label = "每个beta值的统计量",

```

```

xlabel = "Population Size (N)",
ylabel = "Mean Final Infected",
marker = :circle,
linewidth = 2,
)

plot!(
grouped.beta,
grouped.mean_final_inf,
label = "Mean Final Infected",
marker = :square,
)

plot!(
grouped.beta,
grouped.mean_deaths ./ 3000,
label = "Mean Deaths",
marker = :diamond,
)

savefig(plotsdir("beta_scan.png"))

println("Data loaded from data/beta_scan_all.csv to plots/beta_scan")

# ## Parameters
#
# N: Population Size, beta: Infection Rate, gamma: Recovery Rate, delta: Death Rate
# Initial Susceptible: S0, Initial Infected: I0, Initial Recovered: R0, Initial Deaths: D0

```

```
# scan_literate_params.jl scan_literate_params.jl. scan_literate_params.jl scan_literate_params.jl scan_literate_params.jl
# beta scan_literate_params.jl scan_literate_params.jl, scan_literate_params.jl scan_literate_params.jl
# scan_literate_params.jl.
```

4.5 Параметризованная версия второго скрипта

На следующем этапе была подготовлена параметрическая версия второго скрипта, а также сгенерированы notebook и qmd-файлы.

```
[evkuznina@fedora scripts]$ gedit sir_scan_literate.jl
[evkuznina@fedora scripts]$ touch scan_literate_params.jl
[evkuznina@fedora scripts]$ gedit scan_literate_params.jl
[evkuznina@fedora scripts]$ julia --project=. tangle.jl scan_literate_params.jl
[ Info: generating plain script file from "/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scan_literate_params.jl"
[ Info: writing result to "/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scripts/scan_literate_params/scan_literate_params.jl"
✓ Матрица script: /home/evkuznina/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scripts/scan_literate_params/scan_literate_params.jl

[ Info: generating markdown page from "/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scan_literate_params.jl"
[ Info: writing result to "/work/study/2026-1/simulation-modeling/labs/lab04/scripts/markdown/scan_literate_params/scan_literate_params.qmd"
✓ Quarto: /home/evkuznina/work/study/2026-1/simulation-modeling/labs/lab04/scripts/markdown/scan_literate_params/scan_literate_params.qmd

[ Info: generating notebook from "/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scan_literate_params.jl"
[ Info: writing result to "/work/study/2026-1/simulation-modeling/labs/lab04/scripts/notebooks/scan_literate_params/scan_literate_params.ipynb"
✓ Notebook: /home/evkuznina/work/study/2026-1/simulation-modeling/labs/lab04/scripts/notebooks/scan_literate_params/scan_literate_params.ipynb

Готово! Все файлы созданы.
[evkuznina@fedora scripts]$
[evkuznina@fedora scripts]$ ls notebooks/scan_literate_params.jl
ls: невозможно получить доступ к 'notebooks/scan_literate_params.jl': Нет такого файла или каталога
[evkuznina@fedora scripts]$ ls notebooks/scan_literate_params
scan_literate_params.ipynb
[evkuznina@fedora scripts]$ ls markdown/scan_literate_params
scan_literate_params.qmd
[evkuznina@fedora scripts]$ julia --project
julia>
julia> using Pkg
julia> using IJulia
julia> notebook()
notebook = "/usr/bin/jupyter notebook"
[ Info: running "/usr/bin/jupyter notebook"
```

Рисунок 4.12: Создание параметрической версии второго скрипта и генерация файлов

На рисунке показан этап подготовки параметризованной версии второго скрипта.

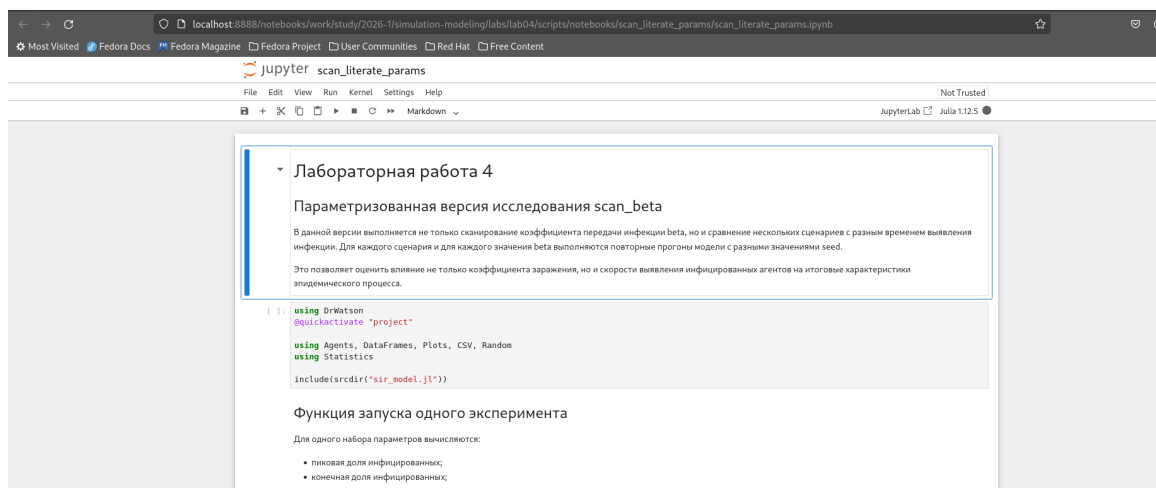


Рисунок 4.13: Notebook параметризованной версии второго скрипта

На рисунке представлен notebook параметризованной версии второго скрипта.

4.5.1 Результаты параметризованного исследования

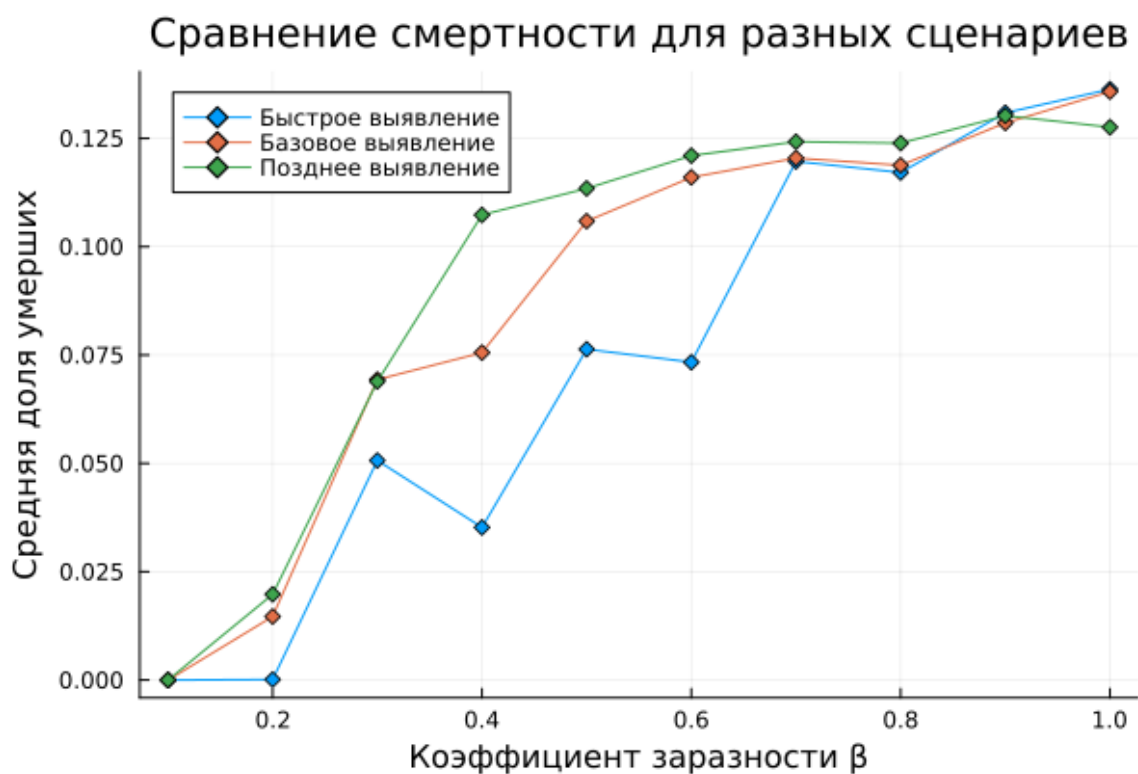


Рисунок 4.14: Сценарное исследование смертности

Сравнение пика эпидемии для разных сценариев

Средняя доля инфицированных

Коэффициент заразности β

Легенда:

- Быстрое выявление (синяя линия)
- Базовое выявление (оранжевая линия)
- Позднее выявление (зеленая линия)

Коэффициент заразности β	Быстрое выявление	Базовое выявление	Позднее выявление
0.0	0.00	0.00	0.00
0.2	0.00	0.00	0.33
0.3	0.67	0.67	0.67
0.4	0.33	0.67	1.00
0.5	0.67	1.00	1.00
0.6	0.67	1.00	1.00
0.7	1.00	1.00	1.00
0.8	1.00	1.00	1.00
0.9	1.00	1.00	1.00
1.0	1.00	1.00	1.00

На рисунке сравниваются значения средней пиковой доли инфицированных в различных сценариях. Это позволяет оценить влияние дополнительных условий моделирования на развитие эпидемии.

```

# 初始化 beta, 将 0 初始化到 1000000 个元素, 0 初始化 1000000 个元素
# 初始化 1000000 个元素. 将 0 初始化到 1000000 个元素, 0 初始化 1000000 个元素 beta
# 初始化 1000000 个元素, 0 初始化 1000000 个元素 seed.
#
# 将 1000000 个元素, 0 初始化到 1000000 个元素, 0 初始化 1000000 个元素, 0 初始化 1000000 个元素
# 将 1000000 个元素, 0 初始化到 1000000 个元素, 0 初始化 1000000 个元素, 0 初始化 1000000 个元素
# 将 1000000 个元素, 0 初始化到 1000000 个元素.

using DrWatson
@quickactivate "project"

using Agents, DataFrames, Plots, CSV, Random
using Statistics

include(sourcedir("sir_model.jl"))

# ## 初始化 1000000 个元素, 0 初始化 1000000 个元素
#
# 将 1000000 个元素, 0 初始化到 1000000 个元素, 0 初始化 1000000 个元素:
#
# - 将 1000000 个元素, 0 初始化到 1000000 个元素;
# - 将 1000000 个元素, 0 初始化到 1000000 个元素;
# - 将 1000000 个元素, 0 初始化到 1000000 个元素;
# - 将 1000000 个元素, 0 初始化到 1000000 个元素.

function run_experiment(p)
beta = p[:beta]
_und = fill(beta, 3)

```



```

    x_det = fill(beta / 10, 3)

model = initialize_sir(;
Ns = p[:Ns],
x_und = x_und,
x_det = x_det,
infection_period = p[:infection_period],
detection_time = p[:detection_time],
death_rate = p[:death_rate],
reinfection_probability = p[:reinfection_probability],
Is = p[:Is],
seed = p[:seed],
n_steps = p[:n_steps],
)

infected_fraction(model) =
count(a.status == :I for a in allagents(model)) / nagents(model)

peak_infected = 0.0

for step = 1:p[:n_steps]
agent_ids = collect(allids(model))
for id in agent_ids
agent = try
model[id]
catch
nothing
end

```

```

if agent != nothing
  sir_agent_step!(agent, model)
end
end

frac = infected_fraction(model)
if frac > peak_infected
  peak_infected = frac
end
end

final_infected = infected_fraction(model)
final_recovered = count(a.status == :R for a in allagents(model)) /
total_deaths = sum(p[:Ns]) - nagents(model)

return (
  peak = peak_infected,
  final_inf = final_infected,
  final_rec = final_recovered,
  deaths = total_deaths,
)
end

# ## [ ] [ ]
#
# [ ] [ ] [ ]:
#
# - [ ] [ ];

```

```

# - [Scenario] [Scenario];
# - [Scenario] [Scenario].

scenario_sets = [
  (name = "[Scenario] [Scenario]", detection_time = 5),
  (name = "[Scenario] [Scenario]", detection_time = 7),
  (name = "[Scenario] [Scenario]", detection_time = 10),
]

# ## [Scenario] [Scenario] beta [Scenario] [Scenario] seed

beta_range = 0.1:0.1:1.0
seeds = [42, 43, 44]

# ## [Scenario] [Scenario] [Scenario]

params_list = []

for scenario in scenario_sets
  for b in beta_range
    for s in seeds
      push!(
        params_list,
        Dict(
          :scenario => scenario.name,
          :beta => b,
          :Ns => [1000, 1000, 1000],
          :infection_period => 14,

```

```

:detection_time => scenario.detection_time,
:death_rate => 0.02,
:reinfection_probability => 0.1,
:Is => [0, 0, 1],
:seed => s,
:n_steps => 100,
),
)
end

# ## Run the experiment

results = []

for params in params_list
  data = run_experiment(params)
  push!(results, merge(params, Dict(pairs(data))))
  println("Scenario = $(params[:scenario]), beta = $(params[:beta]), seed = $(params[:seed])")
end

# ## Save the results

df = DataFrame(results)
CSV.write(datadir("beta_scan_scenarios_all.csv"), df)

# ## Plot the results
#

```

```

# 1000000 100000000 10 1000000 1000000 1 1000000 1000000 beta.

grouped = combine(
  groupby(df, [:scenario, :beta]),
  :peak => mean => :mean_peak,
  :final_inf => mean => :mean_final_inf,
  :final_rec => mean => :mean_final_rec,
  :deaths => mean => :mean_deaths,
)

CSV.write(datadir("beta_scan_scenarios_grouped.csv"), grouped)

# ## 1000000000 1000000 1000 100000000
#
# 10 1000000 1000000 1000000 1000000000 10 100000000 10 1000 100000000.

plot(
  xlabel = "10000000000 1000000000 1",
  ylabel = "1000000 1000 100000000000",
  title = "100000000 1000 10000000 10 1000000 100000000",
  linewidth = 2,
)

for scenario in scenario_sets
  subset_df = grouped[grouped.scenario .== scenario.name, :]
  plot!(
    subset_df.beta,
    subset_df.mean_peak,

```

```

label = scenario.name,
marker = :circle,
)
end

savefig(plotsdir("beta_scan_scenarios_peak.png"))

# ## Figure 1: Peak beta values for different scenarios
#
# Figure 1: Peak beta values for different scenarios
# Figure 1: Peak beta values for different scenarios beta.

plot(
xlabel = "Scenario",
ylabel = "Peak beta",
title = "Peak beta values for different scenarios",
linewidth = 2,
)

for scenario in scenario_sets
subset_df = grouped[grouped.scenario .== scenario.name, :]
plot!(
subset_df.beta,
subset_df.mean_deaths ./ 3000,
label = scenario.name,
marker = :diamond,
)
end

```

```

savefig(plotsdir("beta_scan_scenarios_deaths.png"))

println("XXXXXXXXXX XXXXXXXXXX X data/beta_scan_scenarios_all.csv")
println("XXXXXXXXXX XXXXXXXXXX XXXXXXXXXX X data/beta_scan_scenarios_group")
println("XXXXXXX XXXXXXXXXX X plots/beta_scan_scenarios_peak.png X plots

# ## XXXXX
#
# X XXXXXXXXXX XXXX XXXXXXXXXX X XXXXXXXXXX XXXXXX XX XXXXXXXXXX XXXXXXXXXX
# X XXXXXX XXXXXXXXXX XXXXXXXXXX XXXXXXXXXX. XX XXXXXXXXXX XXXXXXXXXX XXXXXXXXXX
# XXXXXXXXXX detection_time XX XX XXXXXXXXXX X XXXXXXXXXX XX XXXXXXXXXX
# XXXXXXXXXX XXXXXXXXXX XXXXXXXXXX beta.

```

4.6 Скрипт исследования миграции

Третий скрипт посвящён исследованию влияния интенсивности миграции на распространение инфекции между городами.

4.6.1 Обычная версия

Листинг 4.7. Обычная версия скрипта исследования миграции
sir_migration_effect.jl

```

using DrWatson
@quickactivate "project"

using Agents, DataFrames, Plots, CSV, Random
using Statistics

```

```

include(sourcedir("sir_model.jl"))

# 创建迁移矩阵
function create_migration_matrix(C, intensity)
M = ones(C, C) .* intensity / (C - 1)
for i in 1:C
M[i, i] = 1 - intensity
end
return M
end

# 初始化模型
function peak_time(p)
migration_rates = create_migration_matrix(p[:C], p[:migration_inten

model = initialize_sir(;
Ns = p[:Ns],
    _und = p[:_und],
    _det = p[:_det],
infection_period = p[:infection_period],
detection_time = p[:detection_time],
death_rate = p[:death_rate],
reinfection_probability = p[:reinfection_probability],
Is = p[:Is],
seed = p[:seed],
migration_rates = migration_rates,
)

```



```

infected_frac(model) =
count(a.status == :I for a in allagents(model)) / nagents(model)

peak = 0.0
peak_step = 0

for step in 1:p[:n_steps]
agent_ids = collect(allids(model))
for id in agent_ids
agent = try
model[id]
catch
nothing
end

if agent !== nothing
sir_agent_step!(agent, model)
end
end

frac = infected_frac(model)
if frac > peak
peak = frac
peak_step = step
end
end

return (peak_time = peak_step, peak_value = peak)

```

```

end

# 1000000 1000000000000 1000000000
migration_intensities = 0.0:0.1:0.5
seeds = [42, 43, 44]

# 1000000000000 1000000000000
params_list = []

for mig in migration_intensities
  for s in seeds
    push!(
      params_list,
      Dict(
        :migration_intensity => mig,
        :C => 3,
        :Ns => [1000, 1000, 1000],
        :x_und => [0.5, 0.5, 0.5],
        :x_det => [0.05, 0.05, 0.05],
        :infection_period => 14,
        :detection_time => 7,
        :death_rate => 0.02,
        :reinfection_probability => 0.1,
        :Is => [1, 0, 0],
        :seed => s,
        :n_steps => 150,
      ),
    )
  )

```

```

end
end

# 读取数据并计算迁移强度
results = []

for params in params_list
  data = peak_time(params)
  push!(results, merge(params, Dict{String, Any}(pairs(data))))
  println("读取数据并计算迁移强度 \ migration_intensity = $(params[:migration_intensity])")
end

# 将结果写入CSV文件
df = DataFrame(results)
CSV.write(datadir("migration_scan_all.csv"), df)

# 计算迁移强度的平均值
grouped = combine(
  groupby(df, [:migration_intensity]),
  :peak_time => mean => :mean_peak_time,
  :peak_value => mean => :mean_peak_value,
)

# 绘制迁移强度的分布图
plot(
  grouped.migration_intensity,
  grouped.mean_peak_time,
  marker = :circle,

```

```

xlabel = "XXXXXXXXXXXXXXXX XXXXXXXX",
ylabel = "XXXXXX XX XXXXX (XXXX)",
label = "XXXXXX XXXXX",
)

plot!(
grouped.migration_intensity,
grouped.mean_peak_value .* 3000,
marker = :square,
label = "XXXXXXXXXX XXXXXXXXXXXXXXXXXXXX",
)

savefig(plotsdir("migration_effect.png"))

println("XXXXXXXXXXXXXXXX XXXXXXXXX X data/migration_scan_all.csv X plots/migr

```

The screenshot shows a terminal window with the following commands and output:

```

[evkuznin@fedora scripts]$ gedit sir_migration_effect.jl
[evkuznin@fedora scripts]$ julia --project- sir_migration_effect.jl
Завершён эксперимент с migration_intensity = 0.0, seed = 42
Завершён эксперимент с migration_intensity = 0.0, seed = 43
Завершён эксперимент с migration_intensity = 0.0, seed = 44
Завершён эксперимент с migration_intensity = 0.1, seed = 42
Завершён эксперимент с migration_intensity = 0.1, seed = 43
Завершён эксперимент с migration_intensity = 0.1, seed = 44
Завершён эксперимент с migration_intensity = 0.2, seed = 42
Завершён эксперимент с migration_intensity = 0.2, seed = 43
Завершён эксперимент с migration_intensity = 0.2, seed = 44
Завершён эксперимент с migration_intensity = 0.3, seed = 42
Завершён эксперимент с migration_intensity = 0.3, seed = 43
Завершён эксперимент с migration_intensity = 0.3, seed = 44
Завершён эксперимент с migration_intensity = 0.4, seed = 42
Завершён эксперимент с migration_intensity = 0.4, seed = 43
Завершён эксперимент с migration_intensity = 0.4, seed = 44
Завершён эксперимент с migration_intensity = 0.5, seed = 42
Завершён эксперимент с migration_intensity = 0.5, seed = 43
Завершён эксперимент с migration_intensity = 0.5, seed = 44
Вызвана команда в data/migration_scan_all.csv в plots/migration_effect.png
[evkuznin@fedora scripts]$ ls
Manifest.toml scan_iterate_params.jl sir_scan_beta.jl sir_scan_iterate.jl
src sir_iterate.jl sir_iterate_params.jl sir_iterate.jl sir_scan_iterate.jl
scripts sir_migration_effect.jl tangle.jl
test sir_migration_effect.jl
Project.toml sir_run_basic.jl
[evkuznin@fedora scripts]$ touch sir_migration_iterate.jl
[evkuznin@fedora scripts]$ gedit sir_migration_iterate.jl
[evkuznin@fedora scripts]$ julia --project- tangle.jl sir_migration_iterate.jl
Info: generating plain script file from "/work/study/2026-1/simulation-modeling/labs/lab04/scripts/sir_migration_iterate.jl"
Info: writing result to "/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scripts/sir_migration_iterate/sir_migration_iterate.jl"
✓ Чекнут экспорт: /home/evkuznin/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scripts/sir_migration_iterate/sir_migration_iterate.jl
Info: generating markdown page from "/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scripts/sir_migration_iterate.jl"
Info: writing result to "/work/study/2026-1/simulation-modeling/labs/lab04/scripts/markdown/sir_migration_iterate/sir_migration_iterate.qmd"
✓ Quarto: /home/evkuznin/work/study/2026-1/simulation-modeling/labs/lab04/scripts/markdown/sir_migration_iterate/sir_migration_iterate.qmd
Info: generating notebook from "/work/study/2026-1/simulation-modeling/labs/lab04/scripts/scripts/sir_migration_iterate.jl"
Info: writing result to "/work/study/2026-1/simulation-modeling/labs/lab04/scripts/notebooks/sir_migration_iterate/sir_migration_iterate.ipynb"
✓ Notebook: /home/evkuznin/work/study/2026-1/simulation-modeling/labs/lab04/scripts/notebooks/sir_migration_iterate/sir_migration_iterate.ipynb
Готово! Все файлы созданы.
[evkuznin@fedora scripts]$ julia --project
[evkuznin@fedora scripts]$

```

Рисунок 4.16: Создание и выполнение третьего скрипта, подготовка literate-версии

На рисунке показан запуск третьего скрипта и подготовка его literate-версии.

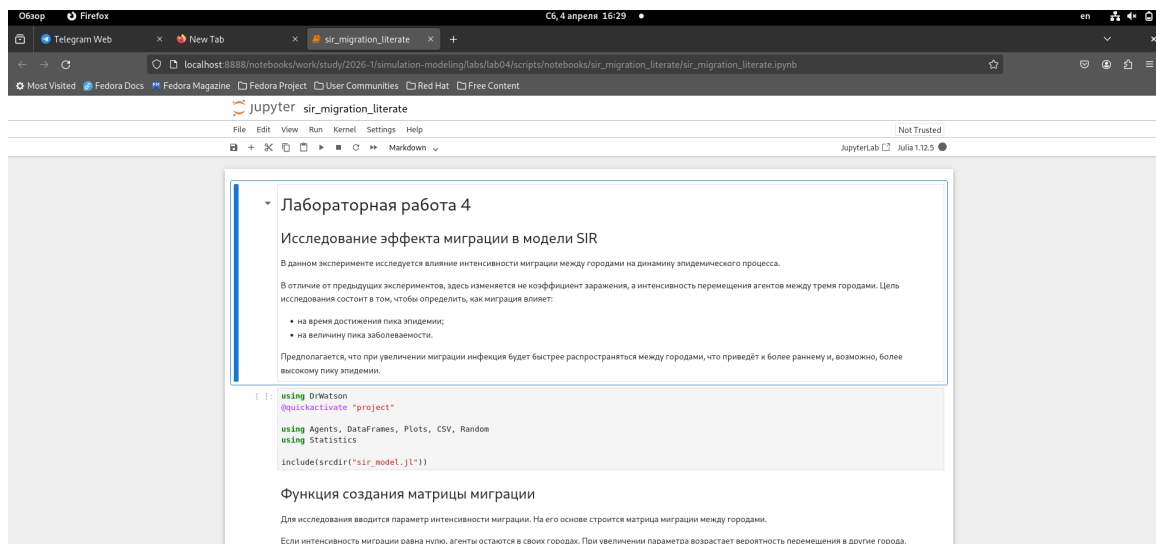


Рисунок 4.17: Notebook literate-версии третьего скрипта

На рисунке представлен notebook literate-версии третьего скрипта.

4.6.2 Результат исследования миграции

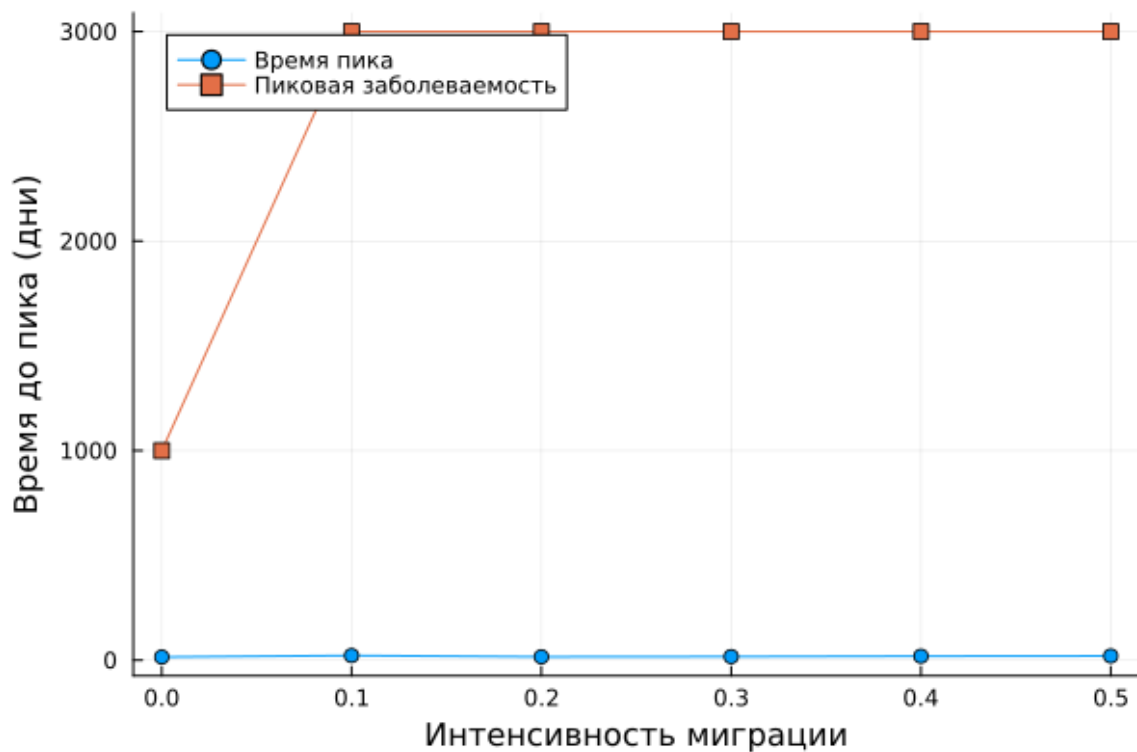


Рисунок 4.18: Влияние интенсивности миграции на динамику эпидемии

На рисунке показано влияние миграции на время достижения пика и на величину пика. При увеличении миграции инфекция распространяется между городами быстрее, что приводит к ускорению эпидемического процесса.

4.6.3 Literate-версия

Листинг 4.8. Literate-версия скрипта исследования миграции

sir_migration_literate.jl

```
# # ██████████ ████████ 4
# ## ████████████████████ ████████████ ████████████ █ ██████████ SIR
#
```

```

# 该函数返回一个字典，其中键为模型名称，值为模型对象。
# 字典的键为模型名称，字典的值为模型对象。
#
# 该函数返回一个字典，其中键为模型名称，值为模型对象。
# 字典的键为模型名称，字典的值为模型对象。
# 字典的键为模型名称，字典的值为模型对象。
# 字典的键为模型名称，字典的值为模型对象。
#
# - 该函数返回一个字典，其中键为模型名称，值为模型对象。
# - 该函数返回一个字典，其中键为模型名称，值为模型对象。
#
# 字典的键为模型名称，字典的值为模型对象。
# 字典的键为模型名称，字典的值为模型对象。
# 字典的键为模型名称，字典的值为模型对象。
# 字典的键为模型名称，字典的值为模型对象。

using DrWatson
@quickactivate "project"

using Agents, DataFrames, Plots, CSV, Random
using Statistics

include(sourcedir("sir_model.jl"))

# ## 该函数返回一个字典，其中键为模型名称，值为模型对象。
#
# 字典的键为模型名称，字典的值为模型对象。
# 字典的键为模型名称，字典的值为模型对象。
#
# 字典的键为模型名称，字典的值为模型对象。
# 字典的键为模型名称，字典的值为模型对象。

```

```

# 1000 1000000000 1000000000 1000000000 1000000000 1000000000 1000000000 1000000000 1000000000 1000000000.

function create_migration_matrix(C, intensity)
M = ones(C, C) .* intensity / (C - 1)
for i in 1:C
M[i, i] = 1 - intensity
end
return M
end

# ## 1000000 1000000000 1000000000 1000000000 1000000000
#
# 1000 1000000000 1000000000 10000000000 10000000000 1000000000000, 1000000 1000000
# 1000000000000:
#
# - 1000000000 1000000000, 1000000 1000000 10000000000000 10000000000 10000000000;
# - 10000000000 1000000 10000000000.

function peak_time(p)
migration_rates = create_migration_matrix(p[:C], p[:migration_inten

model = initialize_sir(;
Ns = p[:Ns],
¶_und = p[:¶_und],
¶_det = p[:¶_det],
infection_period = p[:infection_period],
detection_time = p[:detection_time],
death_rate = p[:death_rate],

```



```

reinfection_probability = p[:reinfection_probability],
Is = p[:Is],
seed = p[:seed],
migration_rates = migration_rates,
)

infected_frac(model) =
count(a.status == :I for a in allagents(model)) / nagents(model)

peak = 0.0
peak_step = 0

for step in 1:p[:n_steps]
agent_ids = collect(allids(model))
for id in agent_ids
agent = try
model[id]
catch
nothing
end

if agent != nothing
sir_agent_step!(agent, model)
end
end

frac = infected_frac(model)
if frac > peak

```



```

Dict(
:migration_intensity => mig,
:C => 3,
:Ns => [1000, 1000, 1000],
:xi_und => [0.5, 0.5, 0.5],
:xi_det => [0.05, 0.05, 0.05],
:infection_period => 14,
:detection_time => 7,
:death_rate => 0.02,
:reinfection_probability => 0.1,
:Is => [1, 0, 0],
:seed => s,
:n_steps => 150,
),
)
end
end

# ## XXXXXXXXXXXXXXXXXXXX
#
# XXX XXXXXXXX XXXXXXX XXXXXXXXXXXXXXX XXXXXXXXXXXXXXX XXXXXXXXXXXXXXX XXXXXXXXXXXXXXX,
# X XXXXXXXXXXXXXXX XXXXXXXXXXXXXXX X XXXXX XXXXXXX.

results = []

for params in params_list
data = peak_time(params)
push!(results, merge(params, Dict(pairs(data))))

```

```

println("XXXXXXXXX XXXXXXXXXXXX ✕ migration_intensity = $(params[:migration_intensity])")
end

# ## XXXXXXXXXXXX XXXXXXXXXXXX
#
# XXXXXXXXXXXX XXXX XXXXXXXXXXXX XXXXXXXXXXXX ✕ CSV-XXXX XXX XXXXXXXXXXXX XXXXXXXX.

df = DataFrame(results)
CSV.write(datadir("migration_scan_all.csv"), df)

# ## XXXXXXXXXXXX XXXXXXXXXXXX
#
# XXX XXXXXXX XXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXX XXXXXXX XXXXXXXXXXXX XXXXXXX XXXXXXXX:
#
# - XXXXXXX XXXXXXXXXXXX XXXX;
# - XXXXXXX XXXX XXXXXXXXXXXXXXXXXXXX.

grouped = combine(
  groupby(df, [:migration_intensity]),
  :peak_time => mean => :mean_peak_time,
  :peak_value => mean => :mean_peak_value,
)

# ## XXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXX
#
# XXXXXXXXXXXX XXXXXXX XXXXXXXXXXXXXXXXXXXX XXXXXXX ✕ XXXX ✕ XXXXXXXXXXXX XXXX
# ✕ XXXXXXXXXXXXXXXXXXXX XXXXXXXX.

```

```

plot(
  grouped.migration_intensity,
  grouped.mean_peak_time,
  marker = :circle,
  xlabel = "XXXXXXXXXXXXXXXX XXXXXXXX",
  ylabel = "XXXXXXXXXX XXXXXXXXXXXXXXX",
  label = "XXXXXX XXXX",
)

plot!(
  grouped.migration_intensity,
  grouped.mean_peak_value .* 3000,
  marker = :square,
  label = "XXXXXXXXXX XXXXXXXXXXXXXXXXXXXX",
)

savefig(plotsdir("migration_effect.png"))

println("XXXXXXXXXXXX XXXXXXXXX X data/migration_scan_all.csv X plots/migr

# ## XXXXX
#
# X XXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXX XXXX XXXXXXXXXXXXXXX XXXXXXXXXXXXXXX XXXXXXXX
# XXXXXXXXXXXXXXX XXXX XXXXXXXXXXXXXXX X XXXXXXXXXXXXXXX XXXX XXXXXXXXXXXXXXXXXXXXXXX XX XXXXXXXXXXXXXXXXXXXXXXX
# XXXXXXXXXXXXXXX XXXXX XXXXXXXXXXXXXXX.
#
# XXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXX XXXXXXXXXXXXXXX X XXXXX XXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXXXXX
# XXXXXXXXXXXXXXX XXXXX XXXXXXXXXXXXXXX, XXX, XXX XXXXXXXXXXXXXXX, XXXXXXXXXXXXXXX XXXXX XX XXXXXXXXXXXXXXX

```

      .

4.7 Параметризованная версия скрипта миграции

```
Окно Терминал С6, 4 апреля 16:33 en
Julia

(evkuzninfedora scripts) touch sir_migration_params.jl
(evkuzninfedora scripts) gedit sir_migration_params.jl
(evkuzninfedora scripts) ls
dirs  markdown  plots  scan_iterate_params.jl  sir_iterate.jl  sir_migration_effect.jl  sir_migration_params.jl  sir_scan_beta.jl  src
Manifest.toml  notebooks  Project.toml  scripts
(evkuzninfedora scripts) julia --project, tangle.jl sir_migration_params.jl
[ Info: generating plain script file from "~/work/study/2026-1/simulation-modeling/Labs/lab04/scripts/sir_migration_params.jl"
[ Info: writing result to "~/work/study/2026-1/simulation-modeling/Labs/lab04/scripts/scripts/sir_migration_params/sir_migration_params.jl"
[ Info: copying ~/home/evkuzninf/work/study/2026-1/simulation-modeling/Labs/lab04/scripts/scripts/sir_migration_params/sir_migration_params.jl
[ Info: generating markdown page from "~/work/study/2026-1/simulation-modeling/Labs/lab04/scripts/sir_migration_params.jl"
[ Info: writing result to "~/work/study/2026-1/simulation-modeling/Labs/lab04/scripts/markdown/sir_migration_params/sir_migration_params.qmd"
[ Info: Quarto: ~/home/evkuzninf/work/study/2026-1/simulation-modeling/Labs/lab04/scripts/markdown/sir_migration_params/sir_migration_params.qmd
[ Info: generating notebook from "~/work/study/2026-1/simulation-modeling/Labs/lab04/scripts/sir_migration_params.jl"
[ Info: writing result to "~/work/study/2026-1/simulation-modeling/Labs/lab04/scripts/notebooks/sir_migration_params/sir_migration_params.ipynb"
Notebook: ~/home/evkuzninf/work/study/2026-1/simulation-modeling/Labs/lab04/scripts/notebooks/sir_migration_params/sir_migration_params.ipynb

Готово! Все файлы созданы.
(evkuzninfedora scripts) ls notebooks/sir_migration_params.jl
ls: не существует такого файла в пути "notebooks/sir_migration_params.jl": No such file or directory
(evkuzninfedora scripts) ls notebooks/sir_migration_params
sir_migration_params.ipynb
(evkuzninfedora scripts) ls markdown/sir_migration_params
sir_migration_params.qmd
(evkuzninfedora scripts) julia --project

Julia 1.12.5 (2026-02-09)
Documentation: https://docs.julialang.org
Type "?>" for help, "j?" for pkg help.
Version 1.12.5 (2026-02-09)
Official https://julialang.org release

julia> using Pkg
julia> using IJulia
julia> notebook()
```

Рисунок 4.19: Создание параметрической версии третьего скрипта

На рисунке показан этап создания параметрической версии третьего скрипта.

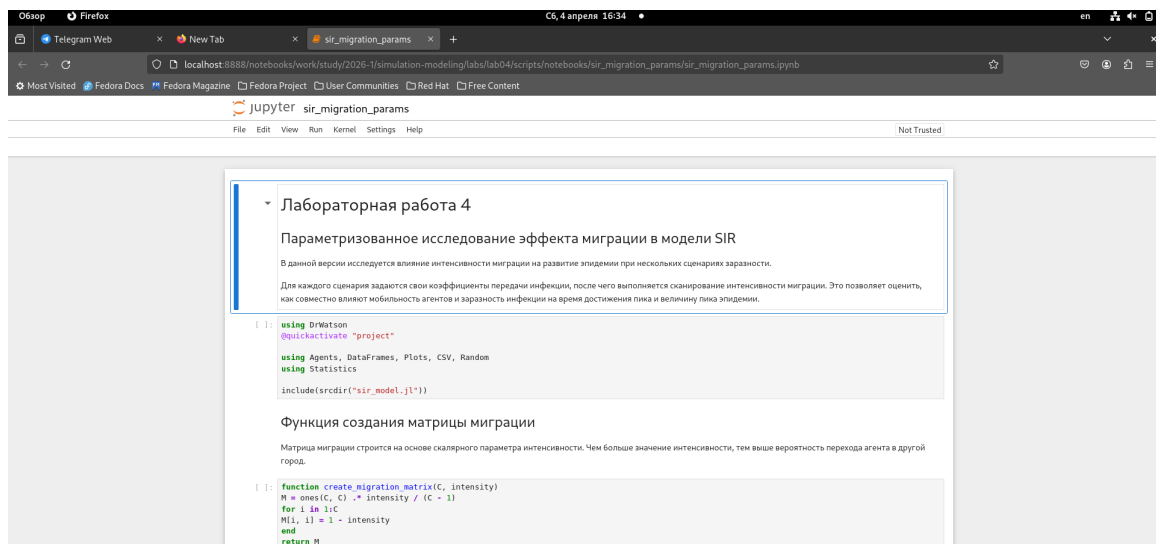


Рисунок 4.20: Notebook параметрической версии третьего скрипта

На рисунке показан notebook параметрической версии третьего скрипта.

4.7.1 Результаты параметризованного исследования миграции

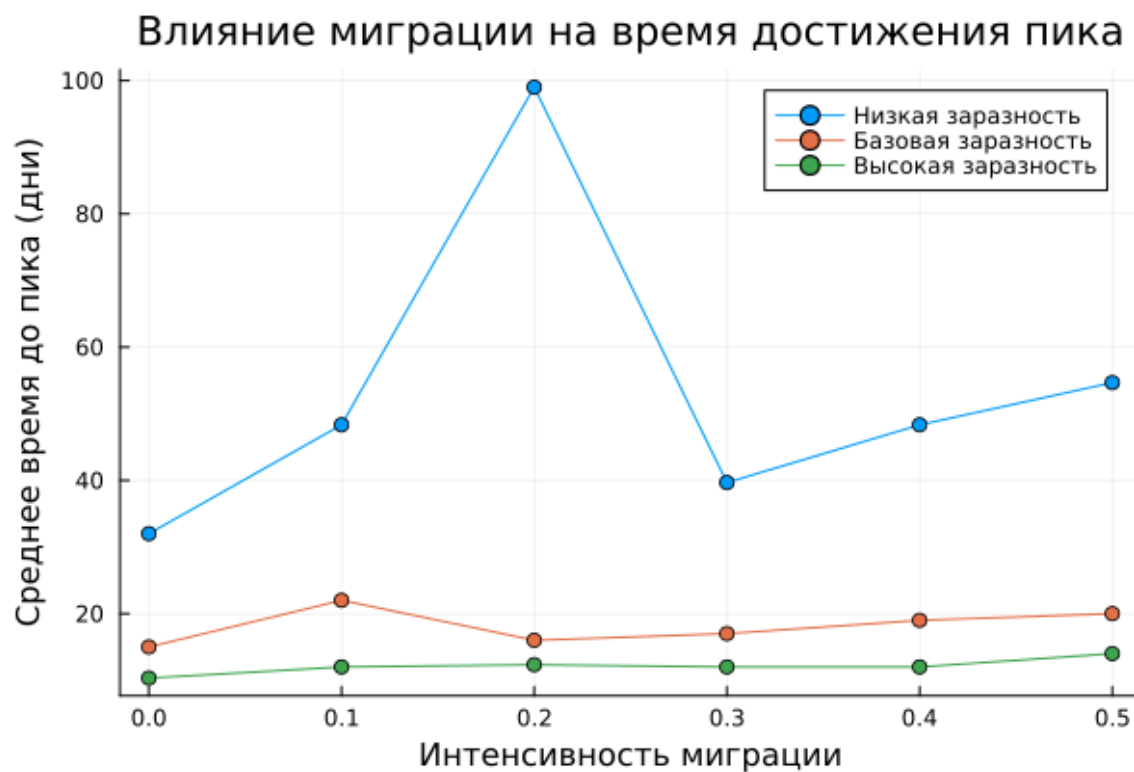


Рисунок 4.21: Сценарное исследование времени до пика при миграции

На рисунке представлена зависимость времени достижения пика от интенсивности миграции в разных сценариях. Это позволяет сравнить скорость распространения инфекции при различных условиях.

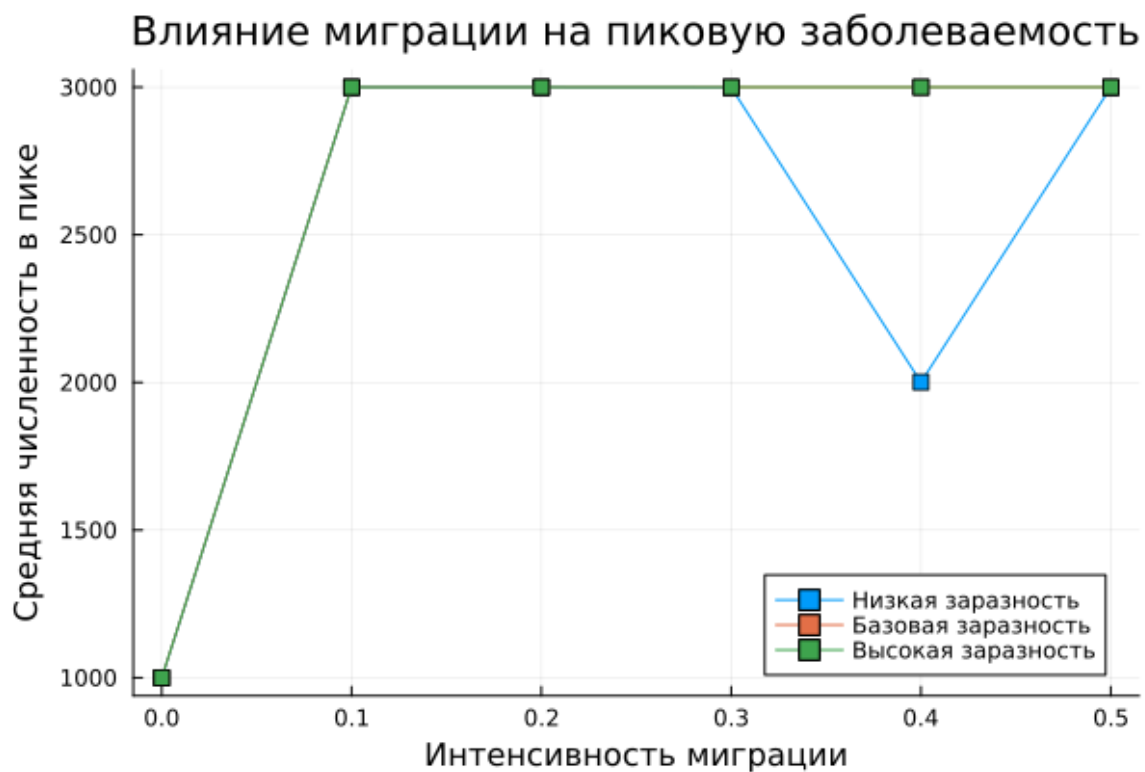


Рисунок 4.22: Сценарное исследование пикового уровня инфицирования при миграции

На рисунке показано, как в разных сценариях меняется величина пика эпидемии. Рост миграции, как правило, способствует более быстрому перемешиванию популяции и усилению распространения инфекции.

Листинг 4.9. Параметризованная версия скрипта исследования миграции
sir_migration_params.jl

```
# # @@@@@@@@@@@@@@@@@ @@@@@@ 4
# ## @@@@@@@@@@@@@@@@@ @@@@@@@@@ @@@@@ @@@@@ @ @@@@@ SIR
#
# @ @@@@@ @@@@@ @@@@@@@@@ @@@@@ @@@@@@@@@@@@@ @@@@@ @ @@@@@@@@@
# @@@@@ @ @ @@@@@@@@@ @@@@@ @@@@@@@@@.
#
#
# @ @ @@@@@ @@@@@ @@@@@ @ @ @@@@@@@@@ @@@@@ @@@@@@,
```

```

# #####  #####  #####  #####  #####  #####.  #####
# #####,  #####  #####  #####  #####  #####
#  #####  #####  #####  #####  #####.

using DrWatson
@quickactivate "project"

using Agents, DataFrames, Plots, CSV, Random
using Statistics

include(sourcedir("sir_model.jl"))

# ## #####  #####  #####  #####
#
# #####  #####  #####  #####  #####  #####  #####.
# #####  #####  #####  #####,  #####  #####  #####
#  #####  #####.

function create_migration_matrix(C, intensity)
M = ones(C, C) .* intensity / (C - 1)
for i in 1:C
M[i, i] = 1 - intensity
end
return M
end

# ## #####  #####  #####  #####
#

```

```

# 1000 1000000000 1000000 10000000000 1000000000000:
#
# - 1000000 10000000000 10000 1000000000;
# - 1000000000 10000 1000000000.

function peak_time(p)
migration_rates = create_migration_matrix(p[:C], p[:migration_inten

model = initialize_sir(;
Ns = p[:Ns],
I_und = p[:I_und],
I_det = p[:I_det],
infection_period = p[:infection_period],
detection_time = p[:detection_time],
death_rate = p[:death_rate],
reinfection_probability = p[:reinfection_probability],
Is = p[:Is],
seed = p[:seed],
migration_rates = migration_rates,
)

infected_frac(model) =
count(a.status == :I for a in allagents(model)) / nagents(model)

peak = 0.0
peak_step = 0

for step in 1:p[:n_steps]

```

```

agent_ids = collect(allids(model))
for id in agent_ids
  agent = try
  model[id]
  catch
  nothing
end

  if agent !== nothing
    sir_agent_step!(agent, model)
  end
end

frac = infected_frac(model)
if frac > peak
  peak = frac
  peak_step = step
end
end

return (peak_time = peak_step, peak_value = peak)
end

# ## [S] [SIR]
#
# [SIR] [SIR] [SIR]:
#
# - [SIR] [SIR];

```



```

results = []

for params in params_list
  data = peak_time(params)
  push!(results, merge(params, Dict(pairs(data))))
  println("XXXXXXXXX = $(params[:scenario]), migration = $(params[:migra
end

# ## XXXXXXXXXXXX XXXXXXXXXXXX
#
# XXXXXXXX XXXXXX XXXXXXXXXXXX XXXX XXXXXXXX.

df = DataFrame(results)
CSV.write(datadir("migration_scenarios_all.csv"), df)

# ## XXXXXXXXXXXX XXXXXXXXXXXX
#
# XXXXXXXX XXXXXXXXXXXX XX XXXXXXXXXXXX XXXXXXXXXXXX XXXXXXXX X XXXXXXXXXXXXXXXX XXXXXXXX.

grouped = combine(
  groupby(df, [:scenario, :migration_intensity]),
  :peak_time => mean => :mean_peak_time,
  :peak_value => mean => :mean_peak_value,
)

CSV.write(datadir("migration_scenarios_grouped.csv"), grouped)

# ## XXXXXXXXXXXX XXXXXXXX XXXXXXXX XX XXXX

```

```

#
#  1.  2.  3.  4.  5.  6.  7.  8.  9.  10.  11.  12.  13.  14.  15.  16.  17.  18.  19.  20.  21.  22.  23.  24.  25.  26.  27.  28.  29.  30.  31.  32.  33.  34.  35.  36.  37.  38.  39.  40.  41.  42.  43.  44.  45.  46.  47.  48.  49.  50.  51.  52.  53.  54.  55.  56.  57.  58.  59.  60.  61.  62.  63.  64.  65.  66.  67.  68.  69.  70.  71.  72.  73.  74.  75.  76.  77.  78.  79.  80.  81.  82.  83.  84.  85.  86.  87.  88.  89.  90.  91.  92.  93.  94.  95.  96.  97.  98.  99.  100.
#  1.  2.  3.  4.  5.  6.  7.  8.  9.  10.  11.  12.  13.  14.  15.  16.  17.  18.  19.  20.  21.  22.  23.  24.  25.  26.  27.  28.  29.  30.  31.  32.  33.  34.  35.  36.  37.  38.  39.  40.  41.  42.  43.  44.  45.  46.  47.  48.  49.  50.  51.  52.  53.  54.  55.  56.  57.  58.  59.  60.  61.  62.  63.  64.  65.  66.  67.  68.  69.  70.  71.  72.  73.  74.  75.  76.  77.  78.  79.  80.  81.  82.  83.  84.  85.  86.  87.  88.  89.  90.  91.  92.  93.  94.  95.  96.  97.  98.  99.  100.

plot(
  xlabel = "Migration Intensity",
  ylabel = "Mean Peak Time (h)",
  title = "Migration Intensity vs Mean Peak Time",
  linewidth = 2,
)

for scenario in scenario_sets
  subset_df = grouped[grouped.scenario == scenario.name, :]
  plot!(
    subset_df.migration_intensity,
    subset_df.mean_peak_time,
    label = scenario.name,
    marker = :circle,
  )
end

savefig(plotsdir("migration_scenarios_peak_time.png"))

# ## 1. 2. 3. 4. 5. 6. 7. 8. 9. 10. 11. 12. 13. 14. 15. 16. 17. 18. 19. 20. 21. 22. 23. 24. 25. 26. 27. 28. 29. 30. 31. 32. 33. 34. 35. 36. 37. 38. 39. 40. 41. 42. 43. 44. 45. 46. 47. 48. 49. 50. 51. 52. 53. 54. 55. 56. 57. 58. 59. 60. 61. 62. 63. 64. 65. 66. 67. 68. 69. 70. 71. 72. 73. 74. 75. 76. 77. 78. 79. 80. 81. 82. 83. 84. 85. 86. 87. 88. 89. 90. 91. 92. 93. 94. 95. 96. 97. 98. 99. 100.
#
#  1.  2.  3.  4.  5.  6.  7.  8.  9.  10.  11.  12.  13.  14.  15.  16.  17.  18.  19.  20.  21.  22.  23.  24.  25.  26.  27.  28.  29.  30.  31.  32.  33.  34.  35.  36.  37.  38.  39.  40.  41.  42.  43.  44.  45.  46.  47.  48.  49.  50.  51.  52.  53.  54.  55.  56.  57.  58.  59.  60.  61.  62.  63.  64.  65.  66.  67.  68.  69.  70.  71.  72.  73.  74.  75.  76.  77.  78.  79.  80.  81.  82.  83.  84.  85.  86.  87.  88.  89.  90.  91.  92.  93.  94.  95.  96.  97.  98.  99.  100.

plot(

```



```

xlabel = "Population Size",
ylabel = "Population Size at Time",
title = "Population Size at Time",
linewidth = 2,
)

for scenario in scenario_sets
subset_df = grouped[grouped.scenario == scenario.name, :]
plot!(
subset_df.migration_intensity,
subset_df.mean_peak_value .* 3000,
label = scenario.name,
marker = :square,
)
end

savefig(plotsdir("migration_scenarios_peak_value.png"))

println("Saving migration_scenarios_all.csv")
println("Saving migration_scenarios_grouped.csv")
println("Saving migration_scenarios_peak_time.png")

# ## Summary
#
# The following table shows the population size at time for each scenario. The population size is calculated as the mean of the population size at time for each scenario. The population size is calculated as the mean of the population size at time for each scenario.
# The population size is calculated as the mean of the population size at time for each scenario. The population size is calculated as the mean of the population size at time for each scenario.
# The population size is calculated as the mean of the population size at time for each scenario. The population size is calculated as the mean of the population size at time for each scenario.

```

4.8 Скрипт оптимизации параметров модели

Четвёртый скрипт решает задачу оптимизации параметров модели SIR.

4.8.1 Обычная версия

Листинг 4.10. Обычная версия скрипта оптимизации

sir_optimize_parameters.jl

```
using DrWatson
@quickactivate "project"

using BlackBoxOptim, Random, Statistics
using Agents
using JLD2

include(sourcedir("sir_model.jl"))

function cost_fast(x)
    infected_frac(model) =
        count(a.status == :I for a in allagents(model)) / nagents(model)

    dead_count(model) = 1500 - nagents(model)

    peak_vals = Float64[]
    dead_vals = Float64[]

    for rep in 1:2
```

```

model = initialize_sir(;
Ns = [500, 500, 500],
I_und = fill(x[1], 3),
I_det = fill(x[1] / 10, 3),
infection_period = 10,
detection_time = round(Int, x[2]),
death_rate = x[3],
reinfection_probability = 0.1,
Is = [0, 0, 1],
seed = 42 + rep,
n_steps = 50,
)

peak_infected = 0.0

for step in 1:50
Agents.step!(model, 1)
frac = infected_frac(model)
if frac > peak_infected
peak_infected = frac
end
end

push!(peak_vals, peak_infected)
push!(dead_vals, dead_count(model) / 1500)
end

mean_peak = mean(peak_vals)

```

```

mean_dead = mean(dead_vals)

return mean_peak + mean_dead
end

result = bboptimize(
cost_fast;
Method = :random_search,
SearchRange = [
(0.1, 1.0),
(3.0, 14.0),
(0.01, 0.1),
],
NumDimensions = 3,
MaxTime = 20,
TraceMode = :silent,
)

best = best_candidate(result)
fitness = best_fitness(result)

println("XXXXXXXXXX XXXXXXXXXXXX:")
println("_und = $(best[1])")
println("XXXXX XXXXXXXXXXXX = $(round(Int, best[2])) XXXX")
println("XXXXXXXXXX = $(best[3])")

println("XXXXXXXX XXXXXXX XXXXXXX: $(fitness)")

```

```
save(
  datadir("optimization_fast_result.jld2"),
  Dict(
    "best" => best,
    "fitness" => fitness,
  ),
)
```

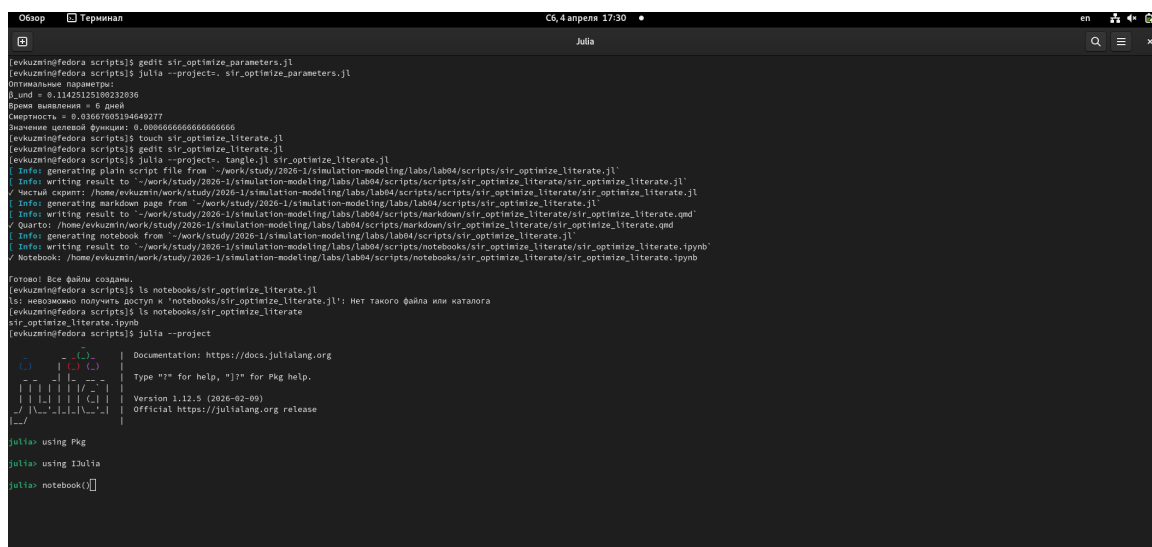


Рисунок 4.23: Создание и выполнение четвёртого скрипта, подготовка literate-версии

На рисунке показан запуск четвёртого скрипта и создание его literate-версии.

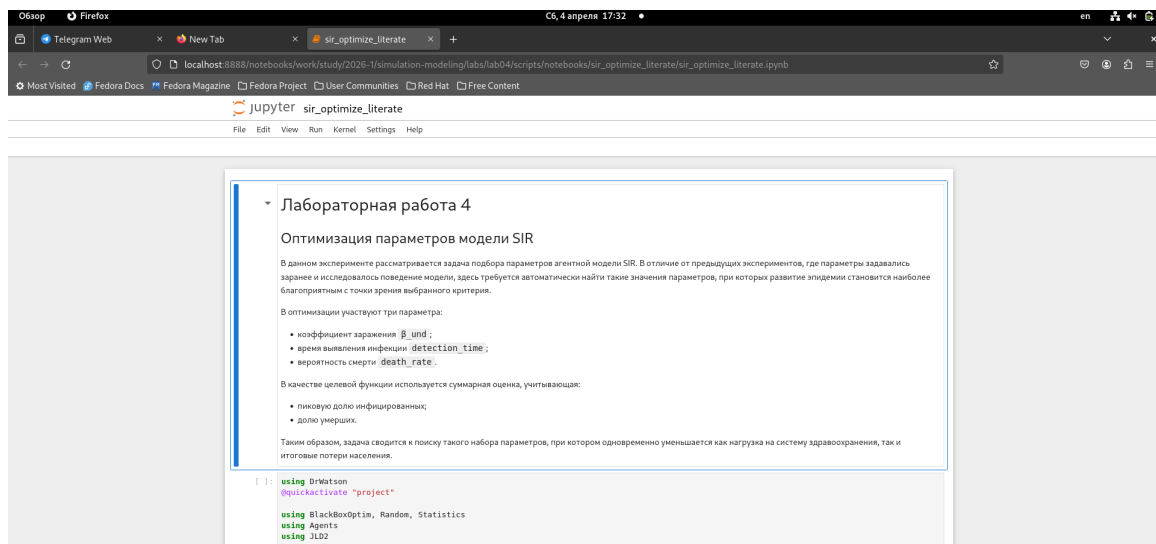


Рисунок 4.24: Notebook literate-версии четвёртого скрипта

На рисунке представлен notebook literate-версии четвёртого скрипта.

4.8.2 Literate-версия

Листинг 4.11. Literate-версия скрипта оптимизации

sir_optimize_iterate.jl

```
# # Лабораторная работа 4
# ## Оптимизация параметров модели SIR
#
# В данном эксперименте рассматривается задача подбора параметров агентной модели SIR. В отличие от предыдущих экспериментов, где параметры задавались
# заранее и исследовалось поведение модели, здесь требуется автоматически найти такие значения параметров, при которых развитие эпидемии становится наиболее
# благоприятным с точки зрения выбранного критерия.
#
# В оптимизации участвуют три параметра:
#
# • коэффициент заражения beta;
# • время выявления инфекции detection_time;
# • вероятность смерти death_rate.
#
# В качестве целевой функции используется суммарная оценка, учитывающая:
#
# • пиковую долю инфицированных;
# • долю умерших.
#
# Таким образом, задача сводится к поиску такого набора параметров, при котором одновременно уменьшается как нагрузка на систему здравоохранения, так и
# итоговые потери населения.
#
# using DrWatson
# @quickactivate "project"
using BlackBoxOptim, Random, Statistics
using Agents
using JLD2
```

```

#
# - 初始化模型参数 `x_und`;
# - 初始化模型参数 `detection_time`;
# - 初始化模型参数 `death_rate`.
#
# 初始化模型参数 `x_und`,
# 初始化模型参数:
#
# - 初始化模型参数 `detection_time`;
# - 初始化模型参数 `death_rate`.
#
# 初始化模型参数, 初始化模型参数 `x_und`,
# 初始化模型参数 `detection_time`,
# 初始化模型参数 `death_rate`.
# 初始化模型参数, 初始化模型参数 `x_und`.

using DrWatson
@quickactivate "project"

using BlackBoxOptim, Random, Statistics
using Agents
using JLD2

include(sourcedir("sir_model.jl"))

# ## 初始化模型参数
#
# 初始化模型参数 `x_und`, 初始化模型参数:
#

```

```

# - `x[1]` -  $\frac{1}{N} \sum_{i=1}^N \frac{1}{\tau_i}$ 
# - `x[2]` -  $\frac{1}{N} \sum_{i=1}^N \frac{1}{\tau_i^2}$ ;
# - `x[3]` -  $\frac{1}{N} \sum_{i=1}^N \frac{1}{\tau_i^3}$ .
#
#  $\frac{1}{N} \sum_{i=1}^N \frac{1}{\tau_i^4}$ 
#  $\frac{1}{N} \sum_{i=1}^N \frac{1}{\tau_i^5}$ .

function cost_fast(x)
  infected_frac(model) =
    count(a.status == :I for a in allagents(model)) / nagents(model)

  dead_count(model) = 1500 - nagents(model)

  peak_vals = Float64[]
  dead_vals = Float64[]

  for rep in 1:2
    model = initialize_sir(;
      Ns = [500, 500, 500],
       $\tau_{und}$  = fill(x[1], 3),
       $\tau_{det}$  = fill(x[1] / 10, 3),
      infection_period = 10,
      detection_time = round{Int, x[2]},
      death_rate = x[3],
      reinfection_probability = 0.1,
      Is = [0, 0, 1],
      seed = 42 + rep,
      n_steps = 50,

```



```

)

peak_infected = 0.0

for step in 1:50
  Agents.step!(model, 1)
  frac = infected_frac(model)
  if frac > peak_infected
    peak_infected = frac
  end
end

push!(peak_vals, peak_infected)
push!(dead_vals, dead_count(model) / 1500)
end

mean_peak = mean(peak_vals)
mean_dead = mean(dead_vals)

return mean_peak + mean_dead
end

# ## [XXXXXXXX] [XXXXXXXXXXXXXXXX]
#
# [XXX] [XXXXXX] [XXXXXXXXXXXXXXXX] [XXXXXXXXXX] [XXXXXXXXXXXXXXXX] [XXXXXXXXXX] [XXXXXXXXXXXXXXXX] [XXXXXXXXXX].
# [XX] [XXXXXXXXXX] [XXXXXXXXXX] [XXXXXX] [XXXXXX] [XXXXXXXXXXXXXXXXXXXXXXXXXXXX] [XXXXXXXXXX] [X] [XXXXXXXXXX]
# [XXX] [XXXXXXXXXXXXXXXXXX] [XXXXXXXXXXXXXXXXXXXXXXXXXXXX] [XXXXXXXXXX] [XXXXXX].

```



```

# ## 训练结果
#
# 训练结果保存在 optimization_fast_result.jld2 文件中
# 训练结果文件名为 optimization_fast_result.jld2。

save(
  datadir("optimization_fast_result.jld2"),
  Dict(
    "best" => best,
    "fitness" => fitness,
  ),
)

# ## 训练结果
#
# 训练结果保存在 optimization_fast_result.jld2 文件中
# 训练结果文件名为 optimization_fast_result.jld2。
# 训练结果文件名为 optimization_fast_result.jld2，
# 训练结果文件名为 optimization_fast_result.jld2。
# 训练结果文件名为 optimization_fast_result.jld2，
# 训练结果文件名为 optimization_fast_result.jld2，
# 训练结果文件名为 optimization_fast_result.jld2，
# 训练结果文件名为 optimization_fast_result.jld2，
# 训练结果文件名为 optimization_fast_result.jld2。

```

4.9 Параметризованная версия скрипта оптимизации

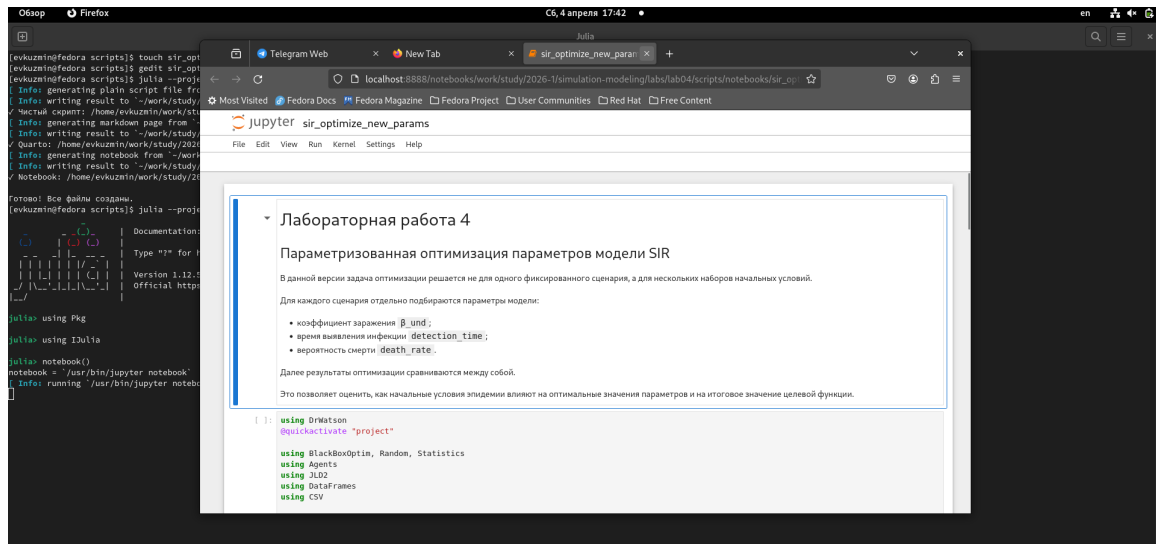


Рисунок 4.25: Генерация файлов и notebook для параметризованной версии четвёртого скрипта

На рисунке показан этап генерации артефактов для параметризованной версии четвёртого скрипта.

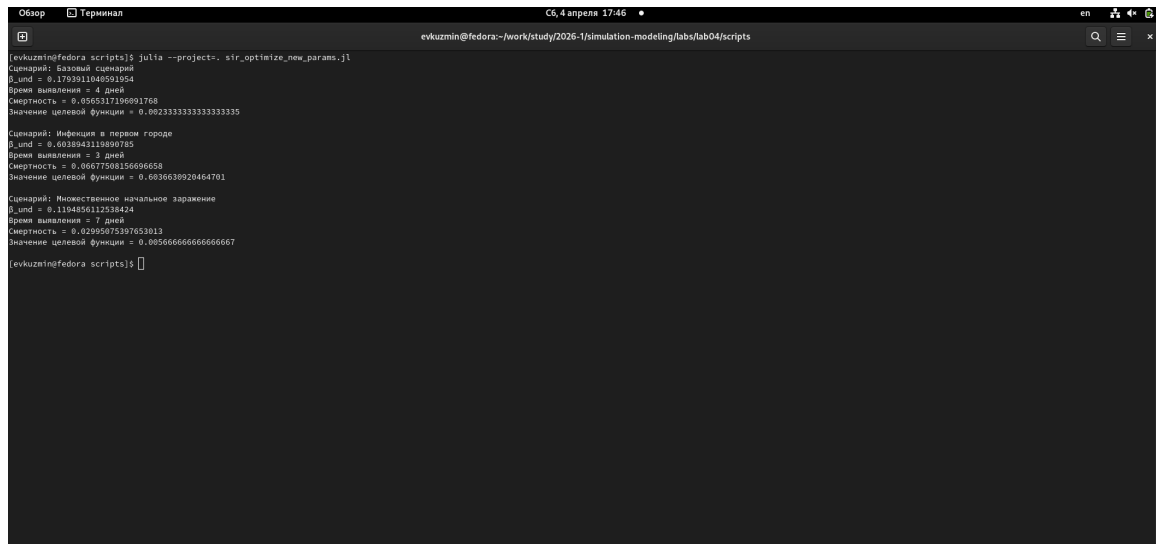


Рисунок 4.26: Выполненная параметризованная версия четвёртого скрипта

На рисунке показан запуск параметризованной версии четвёртого скрипта.

4.9.1 Параметризованная версия

Листинг 4.12. Параметризованная версия скрипта оптимизации

sir_optimize_new_params.jl

```
# # 4
# ## SIR
#
# 4
# , 4
#
# 4:
#
# - `_und`;
# - `detection_time`;
# - `death_rate`.
#
# 4
#
# , 4
# 4.
```

```
using DrWatson
@quickactivate "project"

using BlackBoxOptim, Random, Statistics
using Agents
using JLD2
```

```

using DataFrames
using CSV

include(sourcedir("sir_model.jl"))

# ## 模型 参数
#
# 模型参数:
#
# - 模型参数 在 模型 参数 模型 参数 模型 参数 模型 参数;
# - 模型参数 在 模型 参数 模型 参数 模型 参数 模型 参数;
# - 模型参数 在 模型 参数 模型 参数 模型 参数 模型 参数.

scenario_sets = [
    (name = "模型 参数", Is = [0, 0, 1], Ns = [500, 500, 500]),
    (name = "模型 参数 模型 参数", Is = [1, 0, 0], Ns = [500, 500, 500]),
    (name = "模型 参数 模型 参数", Is = [5, 0, 0], Ns = [500, 500, 500])
]

# ## 模型 参数 模型 参数
#
# 模型 参数 模型 参数 模型 参数 模型 参数 模型 参数 模型 参数, 模型 参数
# 模型 参数 模型 参数 模型 参数 模型 参数.

function make_cost_function(scenario)
    function cost_fast(x)
        infected_frac(model) =
            count(a.status == :I for a in allagents(model)) / nagen
    end
end

```

```

total_population = sum(scenario.Ns)
dead_count(model) = total_population - nagents(model)

peak_vals = Float64[]
dead_vals = Float64[]

for rep in 1:2
    model = initialize_sir(;
        Ns = scenario.Ns,
        x_und = fill(x[1], 3),
        x_det = fill(x[1] / 10, 3),
        infection_period = 10,
        detection_time = round(Int, x[2]),
        death_rate = x[3],
        reinfection_probability = 0.1,
        Is = scenario.Is,
        seed = 42 + rep,
        n_steps = 50,
    )

    peak_infected = 0.0

    for step in 1:50
        Agents.step!(model, 1)
        frac = infected_frac(model)
        if frac > peak_infected
            peak_infected = frac
        end
    end
end

```

```

        end

        push!(peak_vals, peak_infected)
        push!(dead_vals, dead_count(model) / total_population)
    end

    mean_peak = mean(peak_vals)
    mean_dead = mean(dead_vals)

    return mean_peak + mean_dead
end

return cost_fast
end

# ## [XXXXXXXXXX] [XXXXXXXXXXXXXX] [XX] [XX] [XXXXXXXXXX]
#
# [XX] [XXXXXX] [XXXXXXXXXX] [XXXXXXXXXX] [XXXXXXXXXXXXXX] [XXXXXXXXXXXXXX] [ ] [XXXXXXXXXXXXXX]
# [XXXXXXXXXX] [XXXXXXXXXX].

results = DataFrame(
    scenario = String[],
    beta_und = Float64[],
    detection_time = Int[],
    death_rate = Float64[],
    fitness = Float64[],
)

```



```

for scenario in scenario_sets
    cost_fun = make_cost_function(scenario)

    result = bboptimize(
        cost_fun;
        Method = :random_search,
        SearchRange = [
            (0.1, 1.0),
            (3.0, 14.0),
            (0.01, 0.1),
        ],
        NumDimensions = 3,
        MaxTime = 20,
        TraceMode = :silent,
    )

    best = best_candidate(result)
    fitness = best_fitness(result)

    push!(
        results,
        (
            scenario.name,
            best[1],
            round{Int, best[2]},
            best[3],
            fitness,
        ),
    ),

```

```

    )

    println("Scenario: $(scenario.name)")
    println("_und = $(best[1])")
    println("Best fitness = $(round(Int, best[2]))")
    println("Best parameters = $(best[3])")
    println("Best fitness = $(fitness)")
    println()
end

# ## Optimization results

results

# ## Optimization results
#
# Best fitness: 1.0, Best parameters: [0.0, 0.0, 0.0].

CSV.write(datadir("optimization_scenarios.csv"), results)

save(
    datadir("optimization_scenarios.jld2"),
    Dict("results" => results),
)

# ## Optimization results
#
# Best fitness: 1.0, Best parameters: [0.0, 0.0, 0.0].

```

```
# println println println. println println println, println println
# println println println println println println println println
# println println println println println.
```

4.10 Скрипт комплексной визуализации результатов

Пятый скрипт используется для построения составного рисунка по уже рассчитанным данным.

4.10.1 Обычная версия и `literate`-версия

Листинг 4.13. Скрипт комплексной визуализации
`sir_visualize_dynamics.jl`

```
using DrWatson
@quickactivate "project"

using Agents, DataFrames, Plots, CSV

include(scrdir("sir_model.jl"))

# println println println
df = CSV.read(datadir("beta_scan_all.csv"), DataFrame)

# println println: println println println println println
p1 = plot(
```

```

    df.beta,
    df.peak,
    label = "R0",
    xlabel = "t",
    ylabel = "R0(t) (Estimated)",
)

plot!(
    p1,
    df.beta,
    df.final_inf,
    label = "R0(t) (Estimated)",
)

# R0(t) (Estimated): R0(t) (Estimated)
p2 = plot(
    df.beta,
    df.deaths,
    xlabel = "t",
    ylabel = "R0(t) (Estimated)",
    label = "R0(t) (Estimated)",
)

# R0(t) (Estimated): R0(t) (Estimated)
p3 = plot(
    df.beta,
    df.final_rec,
    xlabel = "t",

```

```

        ylabel = "Среднее количество зараженных",
        label = "Среднее количество зараженных",
    )

# Среднее количество зараженных

plot(
    p1,
    p2,
    p3,
    layout = (3, 1),
    size = (800, 900),
)

savefig(plotsdir("comprehensive_analysis.png"))

```

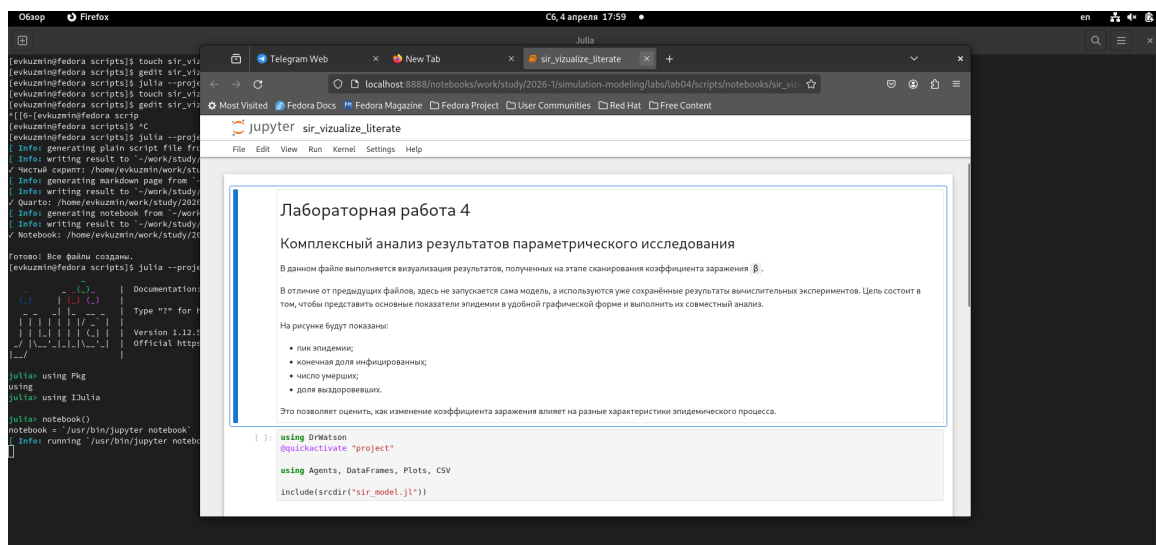


Рисунок 4.27: Создание и выполнение пятого скрипта, генерация literate-версии и notebook

На рисунке показан этап выполнения пятого скрипта и генерации для него notebook и literate-версии.

4.10.2 Результат комплексного анализа

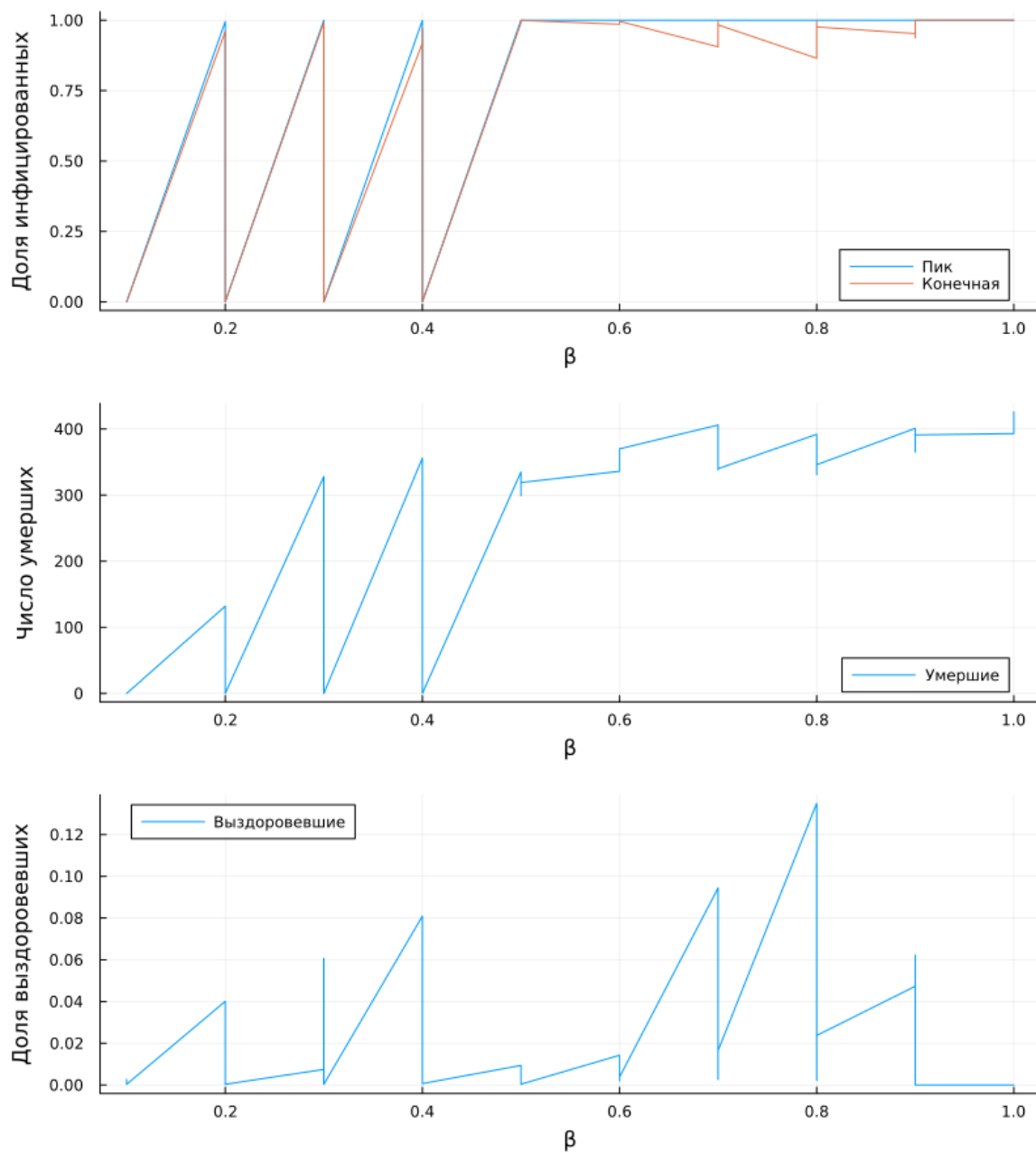


Рисунок 4.28: Комплексный анализ результатов моделирования

На рисунке объединены несколько итоговых зависимостей: пик эпидемии, конечная доля инфицированных, число умерших и доля выздоровевших. Такой

составной график позволяет одновременно анализировать несколько ключевых характеристик модели.

Листинг 4.14. Literate-версия скрипта комплексной визуализации
sir_visualize_literate.jl

```
# # Задача 4
# ## Задача 4.1. Визуализация модели SIR
#
# Цель: Визуализация модели SIR, включая графики численности
# Входные данные: Начальные условия, параметры модели.
#
# Выходные данные: Численности S, I, R, график.
#
# Алгоритм:
# 1. Инициализация параметров и начальных условий.
# 2. Расчет численности S, I, R.
# 3. Визуализация результатов.
#
# Примечания:
# - Используется пакет DrWatson для визуализации.
# - Пакет quickactivate используется для активации проекта.
#
# Используемые пакеты:
# - DrWatson
# - quickactivate
```

```

using Agents, DataFrames, Plots, CSV

include(sourcedir("sir_model.jl"))

# ## Load data
#
# Read data from CSV file
# Read data from CSV file.

df = CSV.read(datadir("beta_scan_all.csv"), DataFrame)

# ## Plot data
#
# Plot data:
#
# - Plot data;
# - Plot data.
#
# Plot data Plot data Plot data Plot data
# Plot data Plot data Plot data.

p1 = plot(
    df.beta,
    df.peak,
    label = "beta",
    xlabel = "t",
    ylabel = "beta peak",
)

```



```

plot!(
    p1,
    df.beta,
    df.final_inf,
    label = "Infected",
)

# ## Infected Susceptible Susceptible
#
# Infected Susceptible Infected Susceptible Infected Susceptible
# Infected Susceptible Infected Infected Susceptible.

p2 = plot(
    df.beta,
    df.deaths,
    xlabel = "t",
    ylabel = "Infected Susceptible",
    label = "Infected",
)

# ## Infected Susceptible Susceptible
#
# Infected Susceptible Infected Infected Infected Infected Infected
# Infected Susceptible Infected Infected Infected.

p3 = plot(
    df.beta,
    df.final_rec,

```

```

xlabel = "x",
ylabel = "x^2 y^2",
label = "x^2 y^2",
)

# ## x^2 y^2
#
# x^2 y^2 x^2 y^2 x^2 y^2
# x^2 y^2 x^2 y^2 x^2 y^2.

plot(
    p1,
    p2,
    p3,
    layout = (3, 1),
    size = (800, 900),
)

savefig(plotsdir("comprehensive_analysis.png"))

# ## x^2 y^2
#
# x^2 y^2 x^2 y^2 x^2 y^2, x^2 y^2 x^2 y^2
# x^2 y^2 x^2 y^2 x^2 y^2.
# x^2 y^2 x^2 y^2 x^2 y^2 x^2 y^2 x^2 y^2
# x^2 y^2 x^2 y^2, x^2 y^2 x^2 y^2 x^2 y^2
# x^2 y^2 x^2 y^2.

```

5 Дополнительные задания

5.1 Дополнительные задания 1–3

Дополнительные задания 1–3 объединены в один скрипт, так как они связаны с базовой динамикой модели, исследованием порога эпидемии и анализом гетерогенности коэффициента заражения.

Листинг 5.1. Скрипт дополнительных заданий 1–3

extra_1_2_3.jl

```
# # Задача 1–3
# ## Задача 1, 2, 3. Исследование порога эпидемии и анализ гетерогенности
#
# В задаче 1–3 исследуется порог эпидемии и анализ гетерогенности:
#
# - Задача 1: Исследование порога эпидемии.
# - Задача 2: Анализ гетерогенности.
# - Задача 3: Анализ гетерогенности.
#
# Задача 1: Исследование порога эпидемии. Задача 2: Анализ гетерогенности.
# Задача 3: Анализ гетерогенности. Задача 4: Анализ гетерогенности SIR.

using DrWatson
@quickactivate "project"
```

```

using Agents, DataFrames, Plots, CSV, Statistics
using JLD2

include(sourcedir("sir_model.jl"))

function main()

# ## 1. 初始化 模型
#
# 初始化 模型 参数 和 变量 的 字典,
# 字典 的 键 为 模型 参数 的 名称, 字典 的 值 为 模型 参数 的 值.
# 字典 的 键 为 模型 变量 的 名称, 字典 的 值 为 模型 变量 的 值.
#
#  $R_0 = \frac{\beta}{\gamma}$ ,  $\gamma = \frac{1}{\text{infection\_period}}$ 

params_basic = Dict(
    :Ns => [1000, 1000, 1000],
    :β_und => [0.5, 0.5, 0.5],
    :β_det => [0.05, 0.05, 0.05],
    :infection_period => 14,
    :detection_time => 7,
    :death_rate => 0.02,
    :reinfection_probability => 0.1,
    :Is => [0, 0, 1],
    :seed => 42,
    :n_steps => 100,
)

```

```

model = initialize_sir(; params_basic...)

times = Int[]
S_vals = Int[]
I_vals = Int[]
R_vals = Int[]

for step in 1:params_basic[:n_steps]
    Agents.step!(model, 1)
    push!(times, step)
    push!(S_vals, susceptible_count(model))
    push!(I_vals, infected_count(model))
    push!(R_vals, recovered_count(model))
end

basic_df = DataFrame(
    time = times,
    susceptible = S_vals,
    infected = I_vals,
    recovered = R_vals,
)

Δ = 1 / params_basic[:infection_period]
R0 = params_basic[:R0_und][1] / Δ

p_basic = plot(
    basic_df.time,
    basic_df.susceptible,

```

```

        label = "S",
        xlabel = "Time",
        ylabel = "Population",
        title = "SIR",
    )
plot!(p_basic, basic_df.time, basic_df.infected, label = "I")
plot!(p_basic, basic_df.time, basic_df.recovered, label = "R")
savefig(p_basic, plotsdir("extra_1_basic_sir.png"))

println("R0 = $(round(R0, digits=3))")

# ## 2. SIR model
#
# The SIR model is a compartmental model that describes the
# dynamics of a population. It is based on the following
# assumptions:
#
# 1. The population is divided into three compartments:
#    Susceptible (S), Infected (I), and Recovered (R).
# 2. The total population is constant.
# 3. The rate of infection is proportional to the product
#    of the number of susceptible and infected individuals.
# 4. The rate of recovery is proportional to the number of
#    infected individuals.
#
# The model is represented by the following system of
# ordinary differential equations:
#
# 
$$\begin{aligned} \frac{dS}{dt} &= -\beta \frac{S}{N} I \\ \frac{dI}{dt} &= \beta \frac{S}{N} I - \gamma I \\ \frac{dR}{dt} &= \gamma I \end{aligned}$$

#
# where  $\beta$  is the transmission rate,  $\gamma$  is the
# recovery rate, and  $N$  is the total population.
#
# The model is solved using the Runge-Kutta method. The
# initial conditions are  $S(0) = N$ ,  $I(0) = 1$ , and
#  $R(0) = 0$ . The parameters are  $\beta = 1$  and
#  $\gamma = 1$ .
#
# The results are plotted as a line graph showing the
# number of individuals in each compartment over time.
#
# The x-axis is labeled "Time" and the y-axis is labeled
# "Population". The legend indicates that the blue line
# represents the Susceptible population, the red line
# represents the Infected population, and the green line
# represents the Recovered population.
#
# The plot shows that the Susceptible population decreases
# over time, the Infected population increases and then
# decreases, and the Recovered population increases.
#
# The total population remains constant at 1.0.
#
# The plot is saved as "extra_1_basic_sir.png".
#
# The results are also printed to the console.
#
# The R0 value is calculated as  $R_0 = \beta / \gamma$ .
#
# The threshold results are stored in a DataFrame.
#
# The DataFrame has two columns: "beta" and "threshold".
#
# The "beta" column contains the values of the transmission
# rate, and the "threshold" column contains the values of
# the threshold.
#
# The DataFrame is created using the following code:
#
# 
$$\text{threshold\_results} = \text{DataFrame}(\text{beta} = \text{Float64}[],$$


```

```

    peak_fraction = Float64[],
)

for beta in beta_range
    model_thr = initialize_sir(;
        Ns = params_basic[:Ns],
        ⍺_und = fill(beta, 3),
        ⍺_det = fill(beta / 10, 3),
        infection_period = params_basic[:infection_period],
        detection_time = params_basic[:detection_time],
        death_rate = params_basic[:death_rate],
        reinfection_probability = params_basic[:reinfection_probabi
        Is = params_basic[:Is],
        seed = params_basic[:seed],
        n_steps = params_basic[:n_steps],
    )

    peak_fraction = 0.0

    for step in 1:params_basic[:n_steps]
        Agents.step!(model_thr, 1)
        frac = infected_count(model_thr) / population_total
        if frac > peak_fraction
            peak_fraction = frac
        end
    end

    push!(threshold_results, (beta, peak_fraction))

```

```

        if threshold_beta == nothing && peak_fraction > 0.05
            threshold_beta = beta
            threshold_peak = peak_fraction
        end
    end

    theoretical_beta = 0

    p_threshold = plot(
        threshold_results.beta,
        threshold_results.peak_fraction,
        label = "XXXXXXXXXX I",
        xlabel = "I",
        ylabel = "XXXXX XXXXXXXXXXXXXXXX",
        title = "XXXXXXXXXXXXX XXXXXXXX XXXXXXXXXXXX",
    )
    hline!(p_threshold, [0.05], label = "XXXXXX 5%")
    vline!(p_threshold, [theoretical_beta], label = "XXXXXXXXXXXXXXXXXXXX XXXXX R0")
    savefig(p_threshold, plotsdir("extra_2_threshold.png"))

    println("XXXXXXXXXXXXXXXXXXXX XXXXXXX beta = $(round(theoretical_beta, digits=3))")
    if threshold_beta != nothing
        println("XXXXXXXXXXXX XXXXXXX beta = $(threshold_beta), XXX = $(round(threshold_peak, digits=3))")
    else
        println("X XXXXXXXXXXXXXXX beta XXXXXXX XX XXXXXXXX")
    end

    CSV.write(datadir("extra_2_threshold_results.csv"), threshold_results)

```



```

# ## 3. 参数初始化
#
# 我们使用字典来初始化参数，字典的键是参数的名称，值是参数的值。
# 字典的键可以是字符串，也可以是整数。字典的值可以是整数、浮点数、列表、字典等。
#
# 字典的初始化格式如下：
#
# - 字典 1: `dict1 = {}`
# - 字典 2: `dict2 = {'key': 'value'}`
# - 字典 3: `dict3 = {'key': value}`

params_hetero = Dict(
    :Ns => [1000, 1000, 1000],
    :xi_und => [0.2, 0.5, 0.8],
    :xi_det => [0.02, 0.05, 0.08],
    :infection_period => 14,
    :detection_time => 7,
    :death_rate => 0.02,
    :reinfection_probability => 0.1,
    :Is => [0, 0, 1],
    :seed => 42,
    :n_steps => 100,
)

model_hetero = initialize_sir(; params_hetero...)

hetero_total = DataFrame(
    time = Int[],

```

```

        infected = Int[],
    )

hetero_city = DataFrame(
    time = Int[],
    city = Int[],
    infected = Int[],
)

for step in 1:params_hetero[:n_steps]
    Agents.step!(model_hetero, 1)

    push!(hetero_total, (step, infected_count(model_hetero)))

    for city in 1:3
        city_inf = count(a.status == :I && a.pos == city for a in a)
        push!(hetero_city, (step, city, city_inf))
    end
end

p_total = plot(
    hetero_total.time,
    hetero_total.infected,
    label = "Susceptible S",
    xlabel = "Time",
    ylabel = "Number of individuals",
    title = "Heterogeneous population: Susceptible S",
)

```


5.1.1 Базовая динамика



Рисунок 5.2: Базовая динамика SIR для дополнительных заданий

На рисунке показана базовая динамика системы SIR для дополнительных заданий. Видно, что эпидемический процесс проходит стадию роста, достигает пика и затем затухает за счёт уменьшения числа восприимчивых и накопления выздоровевших.

5.1.2 Исследование порога

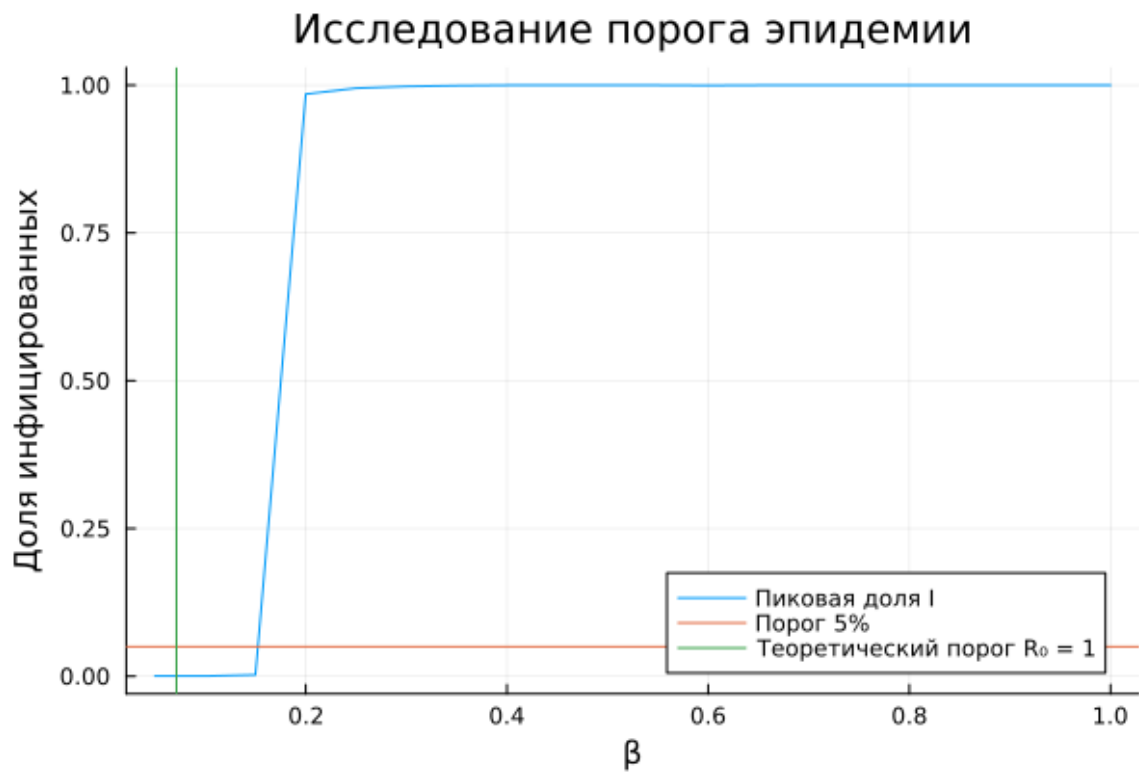


Рисунок 5.3: Исследование порога эпидемии

На рисунке показано исследование эпидемического порога. Горизонтальная линия соответствует уровню 5% популяции, а вертикальная — теоретическому порогу $R_0 = 1$. Сравнение позволяет сопоставить теоретические оценки и наблюдаемые результаты моделирования.

5.1.3 Гетерогенность

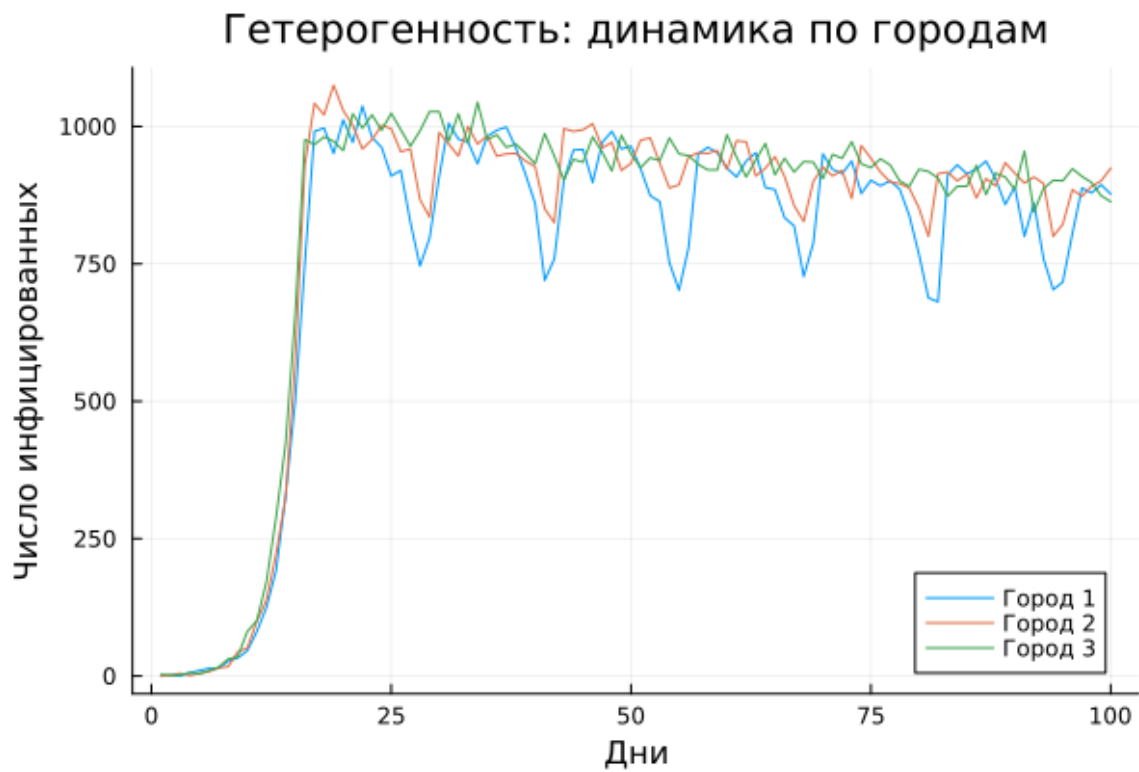


Рисунок 5.4: Динамика инфицированных по городам при гетерогенных параметрах

На рисунке показаны траектории числа инфицированных в отдельных городах при различных значениях β . Более высокая заразность в отдельном городе приводит к более интенсивному локальному развитию эпидемии.

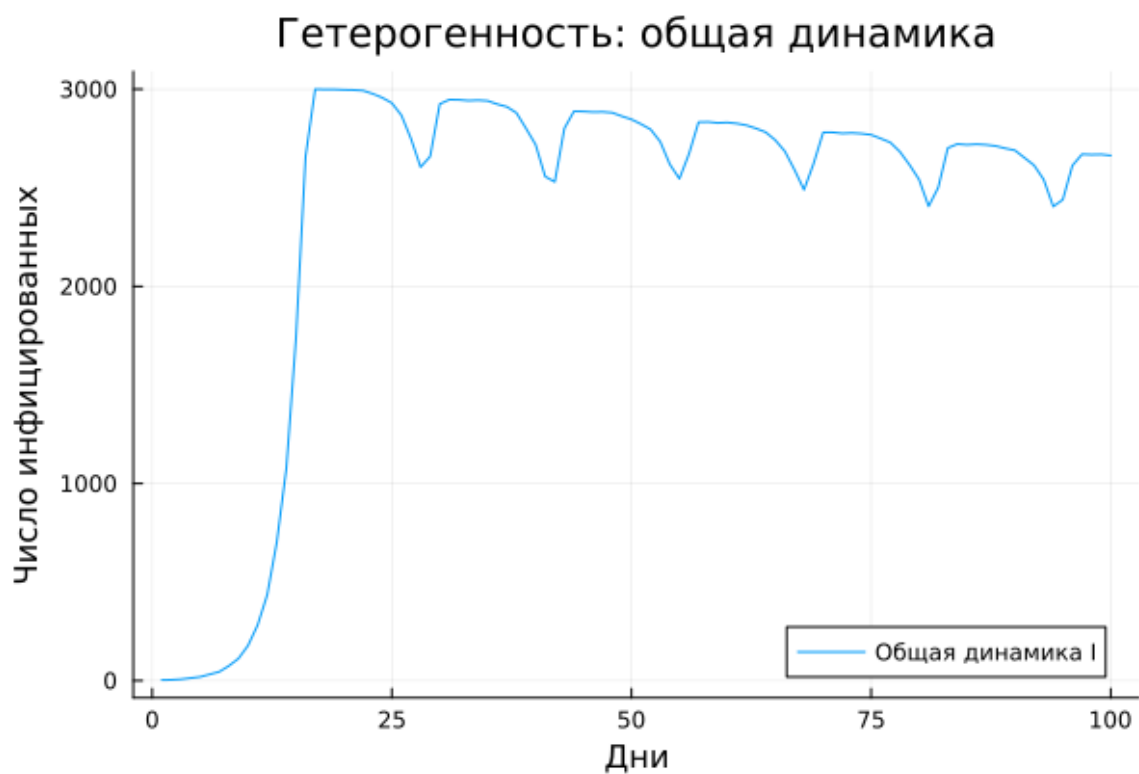


Рисунок 5.5: Общая динамика при гетерогенных параметрах

На рисунке представлена общая динамика инфицированных при гетерогенных параметрах. Введение неоднородности делает поведение системы более сложным по сравнению с однородным случаем.

5.2 Дополнительные задания 4–5

Задания 4 и 5 объединены в один скрипт, так как обе постановки связаны с миграцией и ограничением перемещений между городами.

Листинг 5.2. Скрипт дополнительных заданий 4–5
extra_4_5.jl


```

# # 4-5
# ##
#
# 4-5:
#
# - 4-5 4-5;
# - 4-5.
#
# 4-5, 4-5
# 4-5 4-5.
# 4-5 4-5: 4-5
# 4-5, 4-5.

using DrWatson
@quickactivate "project"

using Agents, DataFrames, Plots, CSV, Statistics
using JLD2

include(sourcedir("sir_model.jl"))

# ##
#
# 4-5.
# 4-5, 4-5
# 4-5 4-5.

function create_migration_matrix(C, intensity)

```

```

    M = ones(C, C) .* intensity / (C - 1)
    for i in 1:C
        M[i, i] = 1 - intensity
    end
    return M
end

# ## create_migration_matrix
#
# create_migration_matrix create_migration_matrix create_migration_matrix:
#
# - create_migration_matrix, create_migration_matrix create_migration_matrix;
# - create_migration_matrix create_migration_matrix.
#
# create_migration_matrix create_migration_matrix create_migration_matrix create_migration_matrix create_migration_matrix.

function peak_time_without_quarantine(p)
    migration_rates = create_migration_matrix(p[:C], p[:migration_i

    model = initialize_sir(;
        Ns = p[:Ns],
         $\tau$ _und = p[: $\tau$ _und],
         $\tau$ _det = p[: $\tau$ _det],
        infection_period = p[:infection_period],
        detection_time = p[:detection_time],
        death_rate = p[:death_rate],
        reinfection_probability = p[:reinfection_probability],
        Is = p[:Is],

```

```

        seed = p[:seed],
        migration_rates = migration_rates,
    )

    infected_frac(model) =
        count(a.status == :I for a in allagents(model)) / nagents(m

peak = 0.0
peak_step = 0

for step in 1:p[:n_steps]
    agent_ids = collect(allids(model))
    for id in agent_ids
        agent = try
            model[id]
        catch
            nothing
        end
        if agent !== nothing
            sir_agent_step!(agent, model)
        end
    end
end

frac = infected_frac(model)
if frac > peak
    peak = frac
    peak_step = step
end

```

```

end

return (peak_time = peak_step, peak_value = peak)
end

# ## 测试函数 与 测试数据
#
# 该函数 测试函数 测试数据 测试数据 测试数据 测试数据:
# 测试 测试 测试数据 测试数据 测试数据 测试数据 测试数据,
# 测试数据 测试 测试数据 测试数据 测试数据.

function peak_time_with_quarantine(p)
    migration_rates = create_migration_matrix(p[:C], p[:migration_i

model = initialize_sir(;
    Ns = p[:Ns],
    I_und = p[:I_und],
    I_det = p[:I_det],
    infection_period = p[:infection_period],
    detection_time = p[:detection_time],
    death_rate = p[:death_rate],
    reinfection_probability = p[:reinfection_probability],
    Is = p[:Is],
    seed = p[:seed],
    migration_rates = copy(migration_rates),
)

infected_frac(model) =

```

```

        count(a.status == :I for a in allagents(model)) / nagents(model)

    peak = 0.0
    peak_step = 0

    quarantine_threshold = p[:quarantine_threshold]
    quarantined_cities = falses(p[:C])

    for step in 1:p[:n_steps]
        for city in 1:p[:C]
            city_population = count(a.pos == city for a in allagents(model))
            city_infected = count(a.status == :I && a.pos == city for a in allagents(model))

            if city_population > 0
                city_frac = city_infected / city_population
                if city_frac > quarantine_threshold && !quarantined_cities[city]
                    model.migration_rates[city, :] .= 0.0
                    model.migration_rates[city, city] = 1.0
                    quarantined_cities[city] = true
                end
            end
        end
    end

    agent_ids = collect(allids(model))
    for id in agent_ids
        agent = try
            model[id]
        catch
            continue
        end
    end
end

```

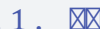
```

        nothing
    end
    if agent != nothing
        sir_agent_step!(agent, model)
    end
end

frac = infected_frac(model)
if frac > peak
    peak = frac
    peak_step = step
end
end

return (peak_time = peak_step, peak_value = peak)
end

function main()

# ##   
#
#      0.0  0.5
#   0.1.      
#      .

migration_intensities = 0.0:0.1:0.5
seeds = [42, 43, 44]

```

```

params_list = []

for mig in migration_intensities
  for s in seeds
    push!(
      params_list,
      Dict(
        :migration_intensity => mig,
        :C => 3,
        :Ns => [1000, 1000, 1000],
        :ϕ_und => [0.5, 0.5, 0.5],
        :ϕ_det => [0.05, 0.05, 0.05],
        :infection_period => 14,
        :detection_time => 7,
        :death_rate => 0.02,
        :reinfection_probability => 0.1,
        :Is => [1, 0, 0],
        :seed => s,
        :n_steps => 150,
        :quarantine_threshold => 0.1,
      ),
    )
  end
end

# ## ██████████ ████████████████████
#
# █████ ██████████ ██████████ ████████████████████ ████████████████████ █████ ██████████:

```



```

        "X XXXXXXXXXXXX",
        res_q.peak_time,
        res_q.peak_value,
    ))

    println("XXXXXXXXX XXXXXXXXXXXX: migration = $(params[:migration_
end

# ## XXXXXXXXXXXX XXXXXXXXXXXX
#
# XXXXXX XXXXXXX XXXXXXXXXXXX XXXXXXXXXXXX X CSV-XXXX.

CSV.write(datadir("extra_4_5_results.csv"), results)

# ## XXXXXXXXXXXX XXXXXXXXXXXX
#
# XXX XXXXXXX XXXXXXX XXXXXXXXXXXXXXX XXXXXXX X XXX XXXXXXX XXXXXXX
# (X XXXXXXXXXXXX X XXX XXXXXXXXXXXX) XXXXXXXXXXXX XXXXXXX XXXXXXXXXXXX:
#
# - XXXXXXX XX XXXX;
# - XXXXXXXXXXX XXXX.

grouped = combine(
  groupby(results, [:migration_intensity, :mode]),
  :peak_time => mean => :mean_peak_time,
  :peak_value => mean => :mean_peak_value,
)

```

```

# ## 物种名称 物种名称 物种名称 物种名称
#
# 物种名称 物种名称 物种名称 物种名称 物种名称 物种名称 物种名称
# 物种名称 物种名称 物种名称 物种名称.

p_time = plot(
    xlabel = "物种名称 物种名称",
    ylabel = "物种名称 物种名称 物种名称",
    title = "物种名称 物种名称: 物种名称 物种名称",
)

for mode in unique(grouped.mode)
    subdf = grouped[grouped.mode == mode, :]
    plot!(p_time, subdf.migration_intensity, subdf.mean_peak_time)
end

savefig(p_time, plotsdir("extra_4_5_peak_time.png"))

# ## 物种名称 物种名称 物种名称 物种名称
#
# 物种名称 物种名称 物种名称 物种名称 物种名称 物种名称 物种名称
# 物种名称 物种名称 物种名称 物种名称 物种名称.

p_peak = plot(
    xlabel = "物种名称 物种名称",
    ylabel = "物种名称 物种名称 物种名称 物种名称",
    title = "物种名称 物种名称: 物种名称 物种名称",
)

```



```
evkuzmin@fedora:~/work/study/2026-1/simulation-modeling/labs/lab04/scripts
[evkuzmin@fedora scripts]$ touch extra_4_5.jl
[evkuzmin@fedora scripts]$ gedit extra_4_5.jl
[evkuzmin@fedora scripts]$ julia --project=. extra_4_5.jl
Завершён эксперимент: migration = 0.0, seed = 42
Завершён эксперимент: migration = 0.0, seed = 43
Завершён эксперимент: migration = 0.0, seed = 44
Завершён эксперимент: migration = 0.1, seed = 42
Завершён эксперимент: migration = 0.1, seed = 43
Завершён эксперимент: migration = 0.1, seed = 44
Завершён эксперимент: migration = 0.2, seed = 42
Завершён эксперимент: migration = 0.2, seed = 43
Завершён эксперимент: migration = 0.2, seed = 44
Завершён эксперимент: migration = 0.3, seed = 42
Завершён эксперимент: migration = 0.3, seed = 43
Завершён эксперимент: migration = 0.3, seed = 44
Завершён эксперимент: migration = 0.4, seed = 42
Завершён эксперимент: migration = 0.4, seed = 43
Завершён эксперимент: migration = 0.4, seed = 44
Завершён эксперимент: migration = 0.5, seed = 42
Завершён эксперимент: migration = 0.5, seed = 43
Завершён эксперимент: migration = 0.5, seed = 44
[evkuzmin@fedora scripts]$
[evkuzmin@fedora scripts]$
```

Рисунок 5.6: Создание и выполнение скрипта дополнительных заданий 4–5

На рисунке показан запуск скрипта дополнительных заданий 4–5.

5.2.1 Влияние миграции и карантина на время до пика

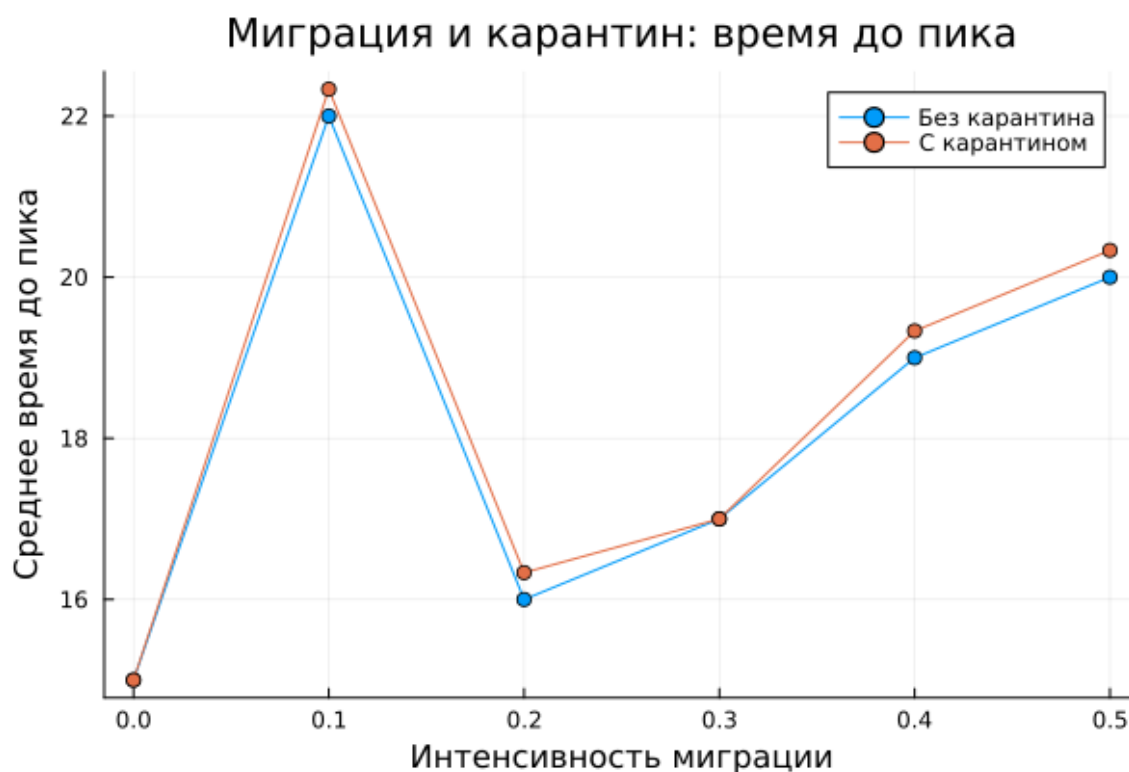


Рисунок 5.7: Сравнение времени достижения пика с карантином и без карантина

На рисунке показано, как меняется среднее время достижения пика при различных значениях миграции. Сравнение режимов с карантином и без карантина позволяет оценить, насколько ограничение перемещений замедляет развитие эпидемии.

5.2.2 Влияние миграции и карантина на величину пика

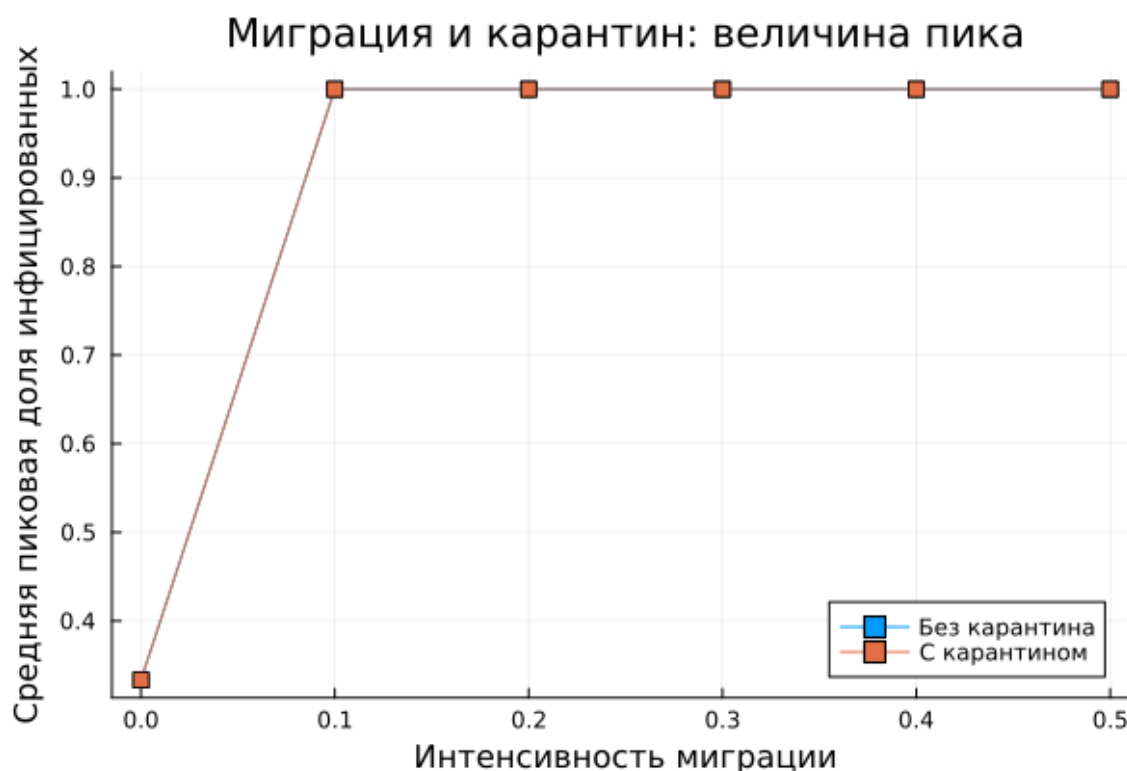


Рисунок 5.8: Сравнение величины пика с карантином и без карантина

На рисунке показана зависимость величины пика эпидемии от интенсивности миграции для режимов с карантином и без карантина. Это позволяет оценить эффективность карантинных мер по снижению максимальной нагрузки.

5.3 Дополнительное задание 6

В дополнительном задании 6 решалась задача оптимизации параметров модели при ограничении на пиковую долю инфицированных.

Листинг 5.3. Скрипт дополнительного задания 6

extra_6.jl

```
# # Задача оптимизации 6
# ## Задача оптимизации параметров SIR
#
# В задаче оптимизации параметров модели SIR необходимо найти оптимальные значения параметров,
# минимизирующие функцию потерь, при этом соблюдая ограничения на пиковую долю инфицированных.
#
# - Минимизация функции потерь;
# - Ограничение на пиковую долю инфицированных (30% от общей популяции).
#
# Функция потерь определяется как сумма квадратов разностей между фактическими и предсказанными значениями.
#
# - Параметры модели: `beta` (коэффициент заражения), `gamma` (коэффициент выздоровления), `delta` (коэффициент смертности);
# - Параметры оптимизации: `detection_time` (время обнаружения), `death_rate` (скорость смертности).
#
# Для решения задачи оптимизации используются пакеты DrWatson, BlackBoxOptim, Random и Statistics.
```

```
using DrWatson
@quickactivate "project"

using BlackBoxOptim, Random, Statistics
```



```

peak_vals = Float64[]
dead_vals = Float64[]

for rep in 1:2
    model = initialize_sir(;
        Ns = [500, 500, 500],
        ⍺_und = fill(x[1], 3),
        ⍺_det = fill(x[1] / 10, 3),
        infection_period = 10,
        detection_time = round(Int, x[2]),
        death_rate = x[3],
        reinfection_probability = 0.1,
        Is = [0, 0, 1],
        seed = 42 + rep,
        n_steps = 50,
    )

    peak_infected = 0.0

    for step in 1:50
        Agents.step!(model, 1)
        frac = infected_frac(model)
        if frac > peak_infected
            peak_infected = frac
        end
    end

    push!(peak_vals, peak_infected)
end

```



```

        push!(dead_vals, dead_frac(model))
    end

    mean_peak = mean(peak_vals)
    mean_dead = mean(dead_vals)

    penalty = 0.0
    if mean_peak > 0.30
        penalty = 10.0 * (mean_peak - 0.30)
    end

    return mean_dead + penalty
end

function main()

# ##  
#
#    ,   
#       .

    result = bboptimize(
        cost_with_constraint;
        Method = :random_search,
        SearchRange = [
            (0.1, 1.0),
            (3.0, 14.0),
            (0.01, 0.1),

```

```
],
    NumDimensions = 3,
    MaxTime = 20,
    TraceMode = :silent,
)

# ## XXXXXXXXXXXX XXXXXXXXXXXX XXXXXXXXXXXX
#
# XXXXX XXXXXXXXXXXX XXXXXXXXXXXX XXXXXXXXXXXX XXXXXXXXXXXX XXXXXXXXXXXX
# X XXXXXXXXXX XXXXXXXXXX XXXXXXXXXX.
```

```
best = best_candidate(result)
fitness = best_fitness(result)

println("XXXXXXXXXXXXXXXX XXXXXXXXXXXX:")
println("_und = $(best[1])")
println("XXXXX XXXXXXXXXXXX = $(round{Int, best[2]}) XXXXX")
println("XXXXXXXXXXXXXXXX = $(best[3])")
println("XXXXXXXXXX XXXXXXXXXX XXXXXXXXXX = $(fitness)")
```

```
# ## XXXXXXXXXXXX XXXXXXXXXX
#
# XXX XXXXXXXXXXXX XXXXXXXXXXXX XXXXXXXXXX XXXXXXXXXXXX XXXXXXX XXXXXXX,
# XXXXXXX XXXXXXXXXX XXXXXXXXXXXX XXX XXXXXXXXXXXXXXXXXXXX X XXXXX XXXXXXXXXX.
```

```
model_check = initialize_sir(;
    Ns = [500, 500, 500],
    _und = fill(best[1], 3),
```

```

    _det = fill(best[1] / 10, 3),
    infection_period = 10,
    detection_time = round(Int, best[2]),
    death_rate = best[3],
    reinfection_probability = 0.1,
    Is = [0, 0, 1],
    seed = 123,
    n_steps = 50,
)

times = Int[]
infected_vals = Float64[]
dead_vals = Float64[]

total_population = 1500
peak_check = 0.0

for step in 1:50
    Agents.step!(model_check, 1)

    current_infected = infected_count(model_check) / total_popu
    current_dead = (total_population - total_count(model_check)

    push!(times, step)
    push!(infected_vals, current_infected)
    push!(dead_vals, current_dead)

    if current_infected > peak_check

```

```

        peak_check = current_infected
    end
end

check_df = DataFrame(
    time = times,
    infected_fraction = infected_vals,
    dead_fraction = dead_vals,
)

println("XXXXXXXXXX XX XXXXXXXXXXXXXXXX = $(round(peak_check, digits=3))")
println("XXXXX 30% XXXXXXXX: $(peak_check <= 0.30)")

# ## XXXXXXXX XXXXXXXX XXXXXXXXXXXXXXXX XXXXXXXX
#
# XX XXXXXXXX XXXXXXXXXXXXXXXX:
#
# - XXXX XXXXXXXXXXXXXXXX;
# - XXXX XXXXXXXX;
# - XXXXXXXXXXXXXXXX XXXXX XXXXXXXXXXXXXXXX 30%.

p = plot(
    check_df.time,
    check_df.infected_fraction,
    label = "XXXXX XXXXXXXXXXXXXXXX",
    xlabel = "XXXX",
    ylabel = "XXXX",
    title = "XXXXXXXXXX XXXXXXXXXXXXXXXX XXXXXXXX",

```

```

    )

    plot!(p, check_df.time, check_df.dead_fraction, label = "Dead Fraction")
    hline!(p, [0.30], label = "Dead Fraction 30%")

    savefig(p, plotsdir("extra_6_optimization_check.png"))

# ## Dead Fraction vs Time
#
# Time (s) vs Dead Fraction, Peak Check vs Time,
# Time (s) vs Dead Fraction vs Peak Check.

CSV.write(datadir("extra_6_check.csv"), check_df)

save(
    datadir("extra_6_optimization_result.jld2"),
    Dict(
        "best" => best,
        "fitness" => fitness,
        "peak_check" => peak_check,
        "check_df" => check_df,
    ),
)

end

main()

```


6 Итоговые выводы

В ходе выполнения лабораторной работы была реализована агентная модель SIR и проведено её подробное исследование.

В работе были выполнены:

- построение базовой модели;
- подготовка обычных и `literate`-версий скриптов;
- генерация `notebook` и `Quarto`-документации;
- параметрические исследования коэффициента заражения;
- анализ миграции между городами;
- решение задачи оптимизации параметров;
- выполнение дополнительных заданий, связанных с порогом эпидемии, гетерогенностью, карантинными мерами и ограниченной оптимизацией.

Полученные результаты показывают, что агентная модель SIR позволяет исследовать широкий круг эпидемиологических сценариев. Изменение параметров модели существенно влияет на время достижения пика, число инфицированных, число умерших и характер распространения инфекции между городами.

7 Список литературы

1. Kermack, W. O., McKendrick, A. G. A Contribution to the Mathematical Theory of Epidemics. Proceedings of the Royal Society A, 1927.
2. Agents.jl — Julia package for agent-based modeling. Available at: <https://juliadynamics.github.io/Agents.jl/>
3. Datseris, G. et al. Agents.jl: A performant and feature-full agent-based modeling software of minimal code complexity. Simulation, 2022.
4. DrWatson.jl — reproducible scientific projects in Julia. Available at: <https://juliadynamics.github.io/DrWatson.jl/>
5. Literate.jl — package for literate programming in Julia. Available at: <https://fredrikekre.github.io/Literate.jl/>
6. BlackBoxOptim.jl — optimization package for Julia. Available at: <https://github.com/robertfeldt/BlackBoxOptim.jl>
7. Методические указания по лабораторной работе №4 по дисциплине «Имитационное моделирование», РУДН, 2026.